```
"""chunk ↔ 源文件行 对齐（只读 JSON 模式，不依赖 fac_rag）。
为什么需要对齐
--------------
切块产物里**没有源行号**：chunk 正文是渲染清洗后的文本（去列表符/粗体/HTML、
按 ``max_chars`` 拆分、表格按行打包）。所以「原文 ↔ 切片」双向高亮只能靠启发式
反查，对齐结果分档：
``exact``        首行在源文件中精确命中
``substr``       首行是某源行的子串（或反之）
``table_rows``   表格块用 ``extras.table_grid.rows`` 逐行反查到源行
``table_label`` 表格块只命中表号/表题
``miss``         对不上，前端置灰、不假高亮
三层策略（与 docs/05-alignment.md 一致）：
1. 条文/段落块：规范化行序列单调扫描，首行剥面包屑，允许回看 2 行；
2. 表格块：``table_grid`` 行反查（去相邻重复单元格 / 去 ``|`` / HTML / 空白），
   超长单行（≥2000 字符，通常是整张 HTML 表）不推进游标，同一行的多行可复用；
3. 兜底：表号/表题定位；再退化为块正文首行扫描。
"""
from __future__ import annotations
import difflib
import html
import re
from typing import Any, Iterable
RE_TAG = re.compile(r"<[^>]+>")
RE_IMG = re.compile(r"!\[[^\]]*\]\([^)]*\)")
RE_PAGE_TAIL = re.compile(r"[.. ·…]{3,}\s*\d+\s*$")
RE_CONT_SUFFIX = re.compile(r"（续）\s*$")
RE_TABLE_NO = re.compile(r"表\s*([A-Za-z]?\s*[.. ]?\s*\d+)")
RE_CAPTION = re.compile(r"^表\s*[A-Za-z]?\s*[.. ]?\s*\d")
DEFAULT_CLEAN_RULES = ("latex_escapes",)
#: 单行 ≥ 该字符数时视为「膨胀后的整张 HTML 表」，同一行可被多个表格块复用。
HUGE_LINE_CHARS = 2000
#: 首行精确/子串都失败时，允许的相似度兜底（上下标 <sup>2</sup>、符号差异等）。
SIMILAR_RATIO = 0.93
SIMILAR_MIN_LEN = 10
SIMILAR_WINDOW = 600
def strip_html(s: str) -> str:
    s = RE_TAG.sub("", s)
    return html.unescape(s)
def strip_media(s: str) -> str:
    """Markdown 图片 → ``[图示]``（与 render.strip_media 对齐）。"""
    return RE_IMG.sub("[图示]", s)
def clean_text(s: str, rules: Iterable[str] = DEFAULT_CLEAN_RULES) -> str:
    """近似 render.clean_text：只做匹配需要的几步（两边同规则，不追求完全一致）。"""
    rules = set(rules or ())
    if "latex_escapes" in rules:
        s = s.replace("\\(", "（").replace("\\)", "）").replace("\\[", "【").replace("\\]", "
】")
    if "md_bold" in rules:
        s = s.replace("**", "")
    if "page_tail" in rules:
```

```
        s = RE_PAGE_TAIL.sub("", s)
    if "continuation_markers" in rules:
        s = RE_CONT_SUFFIX.sub("", s)
    return s
def norm_line(raw: str, rules: Iterable[str] = DEFAULT_CLEAN_RULES) -> str:
    """规范化：去 HTML / 图片占位 / 面包屑 → 去全部空白（两边同规则）。"""
    s = strip_html(strip_media(str(raw).strip()))
    if " > " in s:  # 标准式渲染给 unit.lines[0] 注入的面包屑前缀
        s = s.split(" > ")[-1]
    s = clean_text(s, rules)
    return "".join(s.split())
def norm_pipe(raw: str) -> str:
    s = strip_html(str(raw))
    return "".join(s.replace("|", "").split())
def _flatten(cells: Any) -> list[str]:
    flat: list[str] = []
    def walk(x: Any) -> None:
        if isinstance(x, (list, tuple)):
            for y in x:
                walk(y)
        elif x is not None:
            flat.append(str(x))
    walk(cells)
    return flat
def _cells(row: Any) -> list[str]:
    return [c for c in ("".join(str(x).split()) for x in _flatten(row)) if c]
def _row_in_line(cells: list[str], line: str) -> bool:
    """按单元格顺序在行内找：rowspan/colspan 会让部分单元格只出现一次，
    顺序匹配失败时退化为“所有单元格都出现在这一行”。"""
    pos = 0
    for cell in cells:
        idx = line.find(cell, pos)
        if idx < 0:
            break
        pos = idx + len(cell)
    else:
        return True
    if len(cells) < 2:
        return False
    return all(cell in line for cell in cells)
def norm_row(cells: Any) -> str:
    return "".join("".join(_flatten(cells)).split())
def row_variants(cells: Any) -> list[str]:
    """grid 行 → 候选规范化字符串（合并单元格会导致相邻重复，去重一份备用）。"""
    flat = _flatten(cells)
    raw = "".join("".join(flat).split())
    dedup: list[str] = []
    for cell in flat:
        if dedup and dedup[-1] == cell:
            continue
```

```
            dedup.append(cell)
        variants = [raw]
        d = "".join("".join(dedup).split())
        if d and d != raw:
            variants.append(d)
        return [v for v in variants if v]
def _is_table_line(raw: str) -> bool:
    """这一行本身是「整张表」（HTML 单行或超长膨胀行）→ 表格块的多行可复用同一行。"""
    if len(raw) >= HUGE_LINE_CHARS:
        return True
    low = raw.lstrip().lower()
    return low.startswith("<table") or ("<table" in low and "<tr" in low)
RE_TOC_LINE = re.compile(r"\.{2,}\s*\d+\s*$")
def _scan(lines: list[str], target: str, start: int, *, allow_substr: bool = True) -> int:
    """从 start 起找第一个等于 target（或包含 target）的行。
    单趟扫描、先到先得：优先保证「源行顺序」，而不是「全局找精确匹配」。
    （例：附录标题在源文里写作 ``# （资料性附录）``，若先全局找精确匹配
    会跳到文末的 ``（资料性附录）``，把后续表格块的游标带偏。）
    精确/子串都失败时，再在窗口内找「相似度足够高」的行（兜底上下标
    ``<sup>2</sup>`` 被清洗掉、全角/半角符号等差异，避免这类块整块 miss）。
    """
    if not target:
        return -1
    n = len(lines)
    toc_target = bool(RE_TOC_LINE.search(target))
    for i in range(start, n):
        cand = lines[i]
        if RE_TOC_LINE.search(cand) and not toc_target:
            # 目录行（``3 术语和定义....1``）里也含标题文字，匹配标题时必须跳过，
            # 否则块会锚到目录而不是真正的章节行。
            continue
        if cand == target:
            return i
        if allow_substr and len(target) >= 2 and target in cand:
            return i
    if len(target) < SIMILAR_MIN_LEN:
        return -1
    best = -1
    best_ratio = 0.0
    for i in range(start, min(n, start + SIMILAR_WINDOW)):
        cand = lines[i]
        if not cand:
            continue
        if RE_TOC_LINE.search(cand) and not toc_target:
            continue
        if abs(len(cand) - len(target)) > max(8, len(target) // 4):
            continue
        ratio = difflib.SequenceMatcher(None, target, cand).ratio()
        if ratio > best_ratio:
            best_ratio, best = ratio, i
```

```
    return best if best_ratio >= SIMILAR_RATIO else -1
def _caption_candidates(ns: list[str], label: str, cursor: int, last: int) -> list[int]:
    """表题行候选：``表X.Y …`` 形式、短行、出现在游标附近或命中区间内。"""
    key = "".join(label.split())
    if not key:
        return []
    lo = max(0, cursor - 2)
    hi = min(len(ns), max(last + 10, cursor + 10))
    return [
        k
        for k in range(lo, hi)
        if key in ns[k] and len(ns[k]) < 80 and RE_CAPTION.match(ns[k])
    ]
def _align_table(
    chunk: dict,
    extras: dict,
    ns: list[str],
    npipe: list[str | None],
    raw_lines: list[str],
    cursor: int,
) -> tuple[list[int] | None, str]:
    grid = (extras or {}).get("table_grid") or {}
    rows = grid.get("rows") or []
    if not rows:
        return None, "miss"
    def match_rows(start: int, stop: int) -> tuple[int | None, int | None, int]:
        j = start
        first: int | None = None
        last: int | None = None
        matched = 0
        for row in rows:
            variants = row_variants(row)
            if not variants:
                continue
            hit = -1
            for nr in variants:
                for k in range(j, stop):
                    if npipe[k] is None:
                        npipe[k] = norm_pipe(raw_lines[k])
                    ok = npipe[k] == nr
                    if not ok and len(nr) >= 6:
                        ok = nr in (npipe[k] or "")
                    if ok:
                        hit = k
                        break
                if hit >= 0:
                    break
            if hit < 0:
                # 兜底：按单元格顺序匹配（rowspan/colspan 会让拼接串对不上整张表）
                cells = _cells(row)
```

```
                if cells:
                    for k in range(j, min(stop, j + 400)):
                        if _row_in_line(cells, ns[k]):
                            hit = k
                            break
            if hit >= 0:
                matched += 1
                if first is None:
                    first = hit
                last = hit
                # 整张表压成一行的源行（HTML 单行 / 超长膨胀行）可被多行复用
                j = hit if _is_table_line(raw_lines[hit]) else hit + 1
        return first, last, matched
    first, last, matched = match_rows(max(0, cursor - 2), len(ns))
    if first is None or last is None:
        return None, "miss"
    label = str((extras or {}).get("table_label") or "")
    title = str((extras or {}).get("table_title") or "")
    anchored_at: int | None = None
    # 行命中跨了不止一条「整表行」，或行数没配齐：用表题窗口整表重匹配
    # （同两张表前几行内容完全相同时，避免把后一张表的行算到前一张表上）
    if label and (last > first or matched < len(rows)):
        for cap in _caption_candidates(ns, label, cursor, last):
            stop = len(ns)
            for k in range(cap + 1, len(ns)):
                if RE_CAPTION.match(ns[k]):
                    stop = k
                    break
            f2, l2, m2 = match_rows(cap + 1, stop)
            if f2 is not None and l2 is not None and m2 == len(rows):
                first, last = f2, l2
                anchored_at = cap
                break
    span = [first, last + 1]
    if anchored_at is not None:
        # 锚定到表题行：span 从表题开始，让原文高亮包含表题
        span[0] = anchored_at
    else:
        # 往上把表号/表题带进来（最多 6 行，且不越过上一块终点）
        for b in range(first - 1, max(cursor - 1, first - 7), -1):
            if b < 0:
                break
            line = ns[b]
            if (label and label.replace(" ", "") in line) or (title and title[:6] in line) or
(
                line.startswith("表") and len(line) < 80
            ):
                span[0] = b
                break
    return span, "table_rows"
```

```
def align_chunks(
    chunks: list[dict],
    source_lines: list[str],
    extras: dict[str, dict] | None = None,
    *,
    rules: Iterable[str] = DEFAULT_CLEAN_RULES,
) -> list[dict]:
    """返回与 ``chunks`` 等长的 ``[{"span": [start, end) | None, "status": str}]``。"""
    extras = extras or {}
    ns = [norm_line(line, rules) for line in source_lines]
    npipe: list[str | None] = [None] * len(ns)
    cursor = 0
    label_span: dict[str, list[int]] = {}
    out: list[dict] = []
    for ch in chunks:
        kind = str(ch.get("chunk_kind") or "")
        ex = extras.get(str(ch.get("id") or "")) or {}
        span: list[int] | None = None
        status = "miss"
        # 编辑器在提交时把界面里的行区间写进侧车（extras["span"]），
        # 这里优先采用它，保证「提交 → 重新加载」后对齐结果与编辑时一致。
        pin = ex.get("span")
        if (
            isinstance(pin, list)
            and len(pin) == 2
            and all(isinstance(v, int) for v in pin)
            and 0 <= pin[0] < pin[1] <= len(ns)
        ):
            span = [pin[0], pin[1]]
            status = "approx"
            cursor = max(cursor, span[1])
        if span is None and kind == "table":
            span, status = _align_table(ch, ex, ns, npipe, source_lines, cursor)
            if span:
                cursor = max(cursor, span[1])
        if span is None:
            tlines = [t for t in (norm_line(x, rules) for x in str(ch.get("chunk") or "").spli
t("\n")) if t]
            first_t = tlines[0] if tlines else ""
            # 手工表格块（段落类型、正文以「表X」表题开头）：按表题锚定，
            # 结束位置收在「下一条表题」之前，避免把后面几张表吞进同一区间。
            if first_t and RE_CAPTION.match(first_t):
                cap = _scan(ns, first_t, max(0, cursor - 2))
                if cap < 0:
                    cap = _scan(ns, first_t, 0)
                if cap >= 0:
                    stop = len(ns)
                    for k in range(cap + 1, len(ns)):
                        if ns[k] and RE_CAPTION.match(ns[k]):
                            stop = k
```

```
                        break
                last = cap
                j = cap + 1
                for t in tlines[1:]:
                    k = j
                    limit = min(stop, j + 200)
                    while k < limit and ns[k] != t and (not t or t not in ns[k]):
                        k += 1
                    if k < limit:
                        last = k
                        j = k + 1
                # 表题锚定也可能与上一块重叠（正文各自从同一张表题开始），
                # 起点至少推进到 cursor，避免同一行被两块同时覆盖。
                start = max(cap, min(cursor, last))
                span = [start, last + 1]
                status = "exact" if ns[cap] == first_t else "substr"
                cursor = max(cursor, span[1])
        if span is None:
            tlines = [t for t in (norm_line(x, rules) for x in str(ch.get("chunk") or "").spli
t("\n")) if t]
            first_t = tlines[0] if tlines else ""
            start = max(0, cursor - 2)
            found = _scan(ns, first_t, start)
            if found < 0 and cursor > 0:
                found = _scan(ns, first_t, 0)
            if found >= 0:
                last = found
                j = found + 1
                for t in tlines[1:]:
                    k = j
                    limit = min(len(ns), j + 200)
                    while k < limit and ns[k] != t and (not t or t not in ns[k]):
                        k += 1
                    if k < limit:
                        last = k
                        j = k + 1
                span = [found, last + 1]
                status = "exact" if ns[found] == first_t else "substr"
                cursor = max(cursor, last + 1)
        if span is None and kind == "table":
            label = str(ex.get("table_label") or "")
            title = str(ex.get("table_title") or "")
            keys = [k.replace(" ", "") for k in (label, title) if len(k) >= 2]
            for i in range(max(0, cursor - 2), len(ns)):
                if any(k in ns[i] for k in keys):
                    span = [i, i + 1]
                    status = "table_label"
                    cursor = max(cursor, i + 1)
                    break
        # 老版本提交只写了 15 字段，extras 里的表网格又按旧 id 挂着；这类
```

```
        # 「内容对不上、grid 也找不到」的表格块最后按表号认领已知区间
        # （同一张表的所有块在源文里通常落在同一行/同一段）。
        label_match = RE_TABLE_NO.search(
            " ".join(
                [
                    str(ch.get("law_num") or ""),
                    str(ex.get("table_label") or ""),
                ]
            )
        )
        label_key = "".join(label_match.group(1).split()) if label_match else ""
        if span is None and kind == "table" and label_key:
            ref = label_span.get(label_key)
            if ref is not None:
                span = list(ref)
                status = "table_label"
        if span is None and kind == "table" and cursor > 0:
            # 同一张超长 HTML 表里，前一块若就落在这张表的单行上，后续行
            # （表格拆出来的块）也归到这一行。
            prev_span = next(
                (item["span"] for item in reversed(out) if item.get("span")), None
            )
            if (
                prev_span
                and len(prev_span) == 2
                and prev_span[1] - prev_span[0] == 1
                and "<table" in str(source_lines[prev_span[0]] or "").lower()
            ):
                span = list(prev_span)
                status = "table_label"
        if span and kind == "table" and label_key:
            label_span.setdefault(label_key, list(span))
        out.append({"span": span, "status": status})
    return out
def summarize_align(aligned: list[dict]) -> dict:
    kinds: dict[str, int] = {}
    for item in aligned:
        status = item.get("status") or "miss"
        kinds[status] = kinds.get(status, 0) + 1
    n_aligned = sum(1 for item in aligned if item.get("span"))
    return {
        "n_aligned": n_aligned,
        "n_miss": len(aligned) - n_aligned,
        "align_kinds": kinds,
    }
def locate_text_span(
    norm_lines: list[str],
    text: str,
    *,
    lo: int = 0,
```

```
    hi: int | None = None,
    rules: Iterable[str] = DEFAULT_CLEAN_RULES,
) -> list[int] | None:
    """在 ``norm_lines[lo:hi]`` 窗口里按行顺序定位一段正文，返回 0-based ``[start, end)``。
    用于拆分 / 手工块补行区间：正文行是渲染清洗后的行，所以先用 ``norm_line``
    同口径规范化，再在窗口内做「首行扫描 + 后续行顺序匹配」，不跨出窗口。
    """
    tlines = [t for t in (norm_line(x, rules) for x in str(text or "").split("\n")) if t]
    if not tlines:
        return None
    n = len(norm_lines)
    hi = n if hi is None else max(0, min(hi, n))
    lo = max(0, min(lo, hi))
    if lo >= hi:
        return None
    first = tlines[0]
    found = -1
    for k in range(lo, hi):
        if norm_lines[k] == first or (len(first) >= 2 and first in norm_lines[k]):
            found = k
            break
    if found < 0:
        return None
    last = found
    j = found + 1
    for t in tlines[1:]:
        k = j
        while k < hi and norm_lines[k] != t and (not t or t not in norm_lines[k]):
            k += 1
        if k < hi:
            last = k
            j = k + 1
    return [found, last + 1]
"""编辑命令：纯函数，输入「视图块列表 + 命令」，输出新块列表。
设计原则
--------
- **不改基线**：输入列表不被修改，返回新列表；基线永远只读。
- **派生字段重算**：``token_len = len(chunk)``、``chunk_hash = sha256(规范化正文)[:16]``，
  与 ``fac_rag.contracts`` 一致（这里不 import fac_rag，保持零依赖）。
- **id 规则**（D10）：拆分/新增取「当前最大 seq + 1」；合并且保留前者 id；
  删除留下 id 空洞，靠撤销栈回退。
- **对齐状态**：改动过正文的块标 ``manual``，不再声称与原文某行精确对应；
  拆分/合并能推出区间时保留 span 供参考，否则置 None。
- 所有函数都是幂等的纯计算：同一条 op 序列重复应用结果一致——
  这是「撤销 = 弹出最后一个 op 后整串重放」的前提。
op 形状
-------
- ``{"op": "split", "id": "...", "at_line": 2}`` 或 ``{"op": "split", "id": "...", "at_char":
40}``
- ``{"op": "merge", "id": "前一块", "with": "后一块"}``
```

```
- ``{"op": "merge_many", "ids": ["...", "...", "..."]}``（连续多块集体合并，保留第一块 id）
- ``{"op": "drop", "id": "..."}``
- ``{"op": "add", "text": "...", "law_num": "...", "chunk_kind": "paragraph", "after_id": "..." | null}``
- ``{"op": "patch", "id": "...", "chunk": "...", "law_num": "...", "chunk_kind": "..."}``（都可选）
"""
from __future__ import annotations
import copy
import hashlib
import re
from difflib import SequenceMatcher
from typing import Any, Callable
from .align import locate_text_span, norm_line
RE_ID = re.compile(r"^.+-(\d{4,})(?:-p(\d+))?$")
#: part 块 id 的 ``-pN`` 后缀（删除单个 part 后要重排编号）。
RE_PART_SUFFIX = re.compile(r"-p\d+$")
#: 条号识别：行首（允许列表符 / 加粗 / 标题记号 / 全角空格）的「第X条」
RE_LAW_NUM = re.compile(r"^\s*(?:[#>*\-+]\s*)*第\s*[〇零一二三四五六七八九十百千两0-9]{1,8}\s*条")
#: 标准式条号识别：行首的「5.4.1」这类数字层级号（至少带一个小数点，避免误吞正文里的普通数字）
RE_CLAUSE_NUM = re.compile(r"^\s*(?:[#>*\-+]\s*)*(\d{1,2}(?:\.\d{1,4}){1,5})(?=\s|\u3000|$|[^\d.])")
#: law_num 本身是否可作为倒推锚点（第X条 / 5.4.1 / 1）
RE_ANCHOR = re.compile(r"^(?:第\s*[〇零一二三四五六七八九十百千两0-9]{1,8}\s*条|\d{1,2}(?:\.\d{1,4}){1,5}|\d{1,2})$")
CHUNK_KINDS = frozenset({"clause", "table", "title", "front", "appendix", "stub", "paragraph"})
DEFAULT_LAW_INVALID = "2030-12-12"
#: 新增块没有条号时的占位（契约要求 law_num 非空）
NEW_LAW_NUM = "（新增）"
#: 拆分/新增时从原块继承的 extras 键（表格网格之类不跟过去）
INHERITED_EXTRAS = ("clause", "region", "ref_title", "num_cn")
#: 视图层附加字段（不属于 15 字段契约，操作时原样携带）
VIEW_KEYS = ("extras", "span", "align", "manual")
class OpError(ValueError):
    """编辑命令不合法（400）。"""
class OpNotFound(OpError):
    """要操作的块不存在（404）。"""
# ---------------------------------------------------------------------------
# 纯函数：正文规范化与派生字段（与 fac_rag.contracts 保持一致）
# ---------------------------------------------------------------------------
def normalize_text(text: Any) -> str:
    """统一换行、去掉每行尾空白；正文内容本身不动。"""
    text = str(text or "").replace("\r\n", "\n").replace("\r", "\n")
    lines = [line.rstrip() for line in text.split("\n")]
    while lines and not lines[0]:
        lines.pop(0)
    while lines and not lines[-1]:
        lines.pop()
```

```
    return "\n".join(lines)
def chunk_hash(text: Any) -> str:
    """正文指纹（16 位十六进制），与 FAC-RAG 契约一致。"""
    normalized = "\n".join(line.rstrip() for line in str(text or "").strip().splitlines())
    return hashlib.sha256(normalized.encode("utf-8")).hexdigest()[:16]
def estimate_token_len(text: Any) -> int:
    """字符数近似 token 数。"""
    return len(str(text or ""))
def _derive(chunk: dict, *, text: str | None = None) -> dict:
    """写入正文并重算派生字段；返回同一个 dict。"""
    if text is not None:
        chunk["chunk"] = normalize_text(text)
    else:
        chunk["chunk"] = normalize_text(chunk.get("chunk"))
    chunk["token_len"] = estimate_token_len(chunk["chunk"])
    chunk["chunk_hash"] = chunk_hash(chunk["chunk"])
    try:
        part_index = int(chunk.get("part_index") or 1)
        part_total = int(chunk.get("part_total") or 1)
    except (TypeError, ValueError) as exc:
        raise OpError(f"part 编号不是整数：{exc}") from None
    chunk["part_index"] = part_index
    chunk["part_total"] = max(part_total, part_index)
    return chunk
def _require_text(chunk: dict, what: str) -> None:
    if not str(chunk.get("chunk") or "").strip():
        raise OpError(f"{what}后正文为空，请换一个切分位置")
def _copy(chunk: dict) -> dict:
    return copy.deepcopy(chunk)
def detect_law_num(text: Any) -> str:
    """从一段正文的前几行里识别条号；识别不到返回空串。
    两种样式都认：
    - 法规式「第二条 …」；
    - 标准式「5.4.1 现场应…」（至少一个层级点，避免把正文里的「2019年」之类当条号）。
    用于拆分时自动推断下半块条号：合并过的块再拆开，下半块往往以
    「第二条 …」或「5.4.2 …」开头，不该继续沿用上半块的条号。
    """
    for line in str(text or "").split("\n")[:4]:
        match = RE_LAW_NUM.match(line)
        if match:
            return re.sub(r"\s+", "", match.group(0))
        match = RE_CLAUSE_NUM.match(line)
        if match:
            return match.group(1)
    return ""
def _parse_span(value: Any) -> list[int] | None:
    """解析可选的行区间 span（1-based 闭区间），非法时抛 OpError。"""
    if value is None:
        return None
    if not isinstance(value, (list, tuple)) or len(value) != 2:
```

```
        raise OpError("span 必须是 [起始行, 结束行] 两个整数")
    try:
        start = int(value[0])
        end = int(value[1])
    except (TypeError, ValueError):
        raise OpError("span 的行号不是整数") from None
    if start < 1 or end < 1:
        raise OpError("span 的行号必须从 1 开始")
    if start > end:
        raise OpError(f"span 起始行 {start} 大于结束行 {end}")
    return [start, end]
# ---------------------------------------------------------------------------
# 基础查询
# ---------------------------------------------------------------------------
def find_index(chunks: list[dict], chunk_id: Any) -> int:
    chunk_id = str(chunk_id or "")
    for index, chunk in enumerate(chunks):
        if str(chunk.get("id")) == chunk_id:
            return index
    raise OpNotFound(f"找不到切片：{chunk_id}")
def max_seq(chunks: list[dict]) -> int:
    """当前块 id 里的最大序号（找不到数字时按 0）。"""
    best = 0
    for chunk in chunks:
        match = RE_ID.match(str(chunk.get("id") or ""))
        if match:
            best = max(best, int(match.group(1)))
    return best
def next_id(chunks: list[dict], law_name: str, reserved=None) -> str:
    """新块 id = law_name-(max_seq+1)，并保证不撞已有 id。
    ``reserved`` 用于把「本次编辑前存在过的 id」也算进来：删除后再新增
    不会复用已删块的编号（否则 diff 会把新块误判成旧块被修改）。
    """
    used = {str(chunk.get("id")) for chunk in chunks}
    seq = max_seq(chunks) + 1
    if reserved:
        seq = max(seq, max_seq([{"id": cid} for cid in reserved]) + 1)
        used |= {str(cid) for cid in reserved}
    while True:
        candidate = f"{law_name}-{seq:04d}"
        if candidate not in used:
            return candidate
        seq += 1
def _renumber(chunks: list[dict], law_name: str) -> list[dict]:
    """按当前顺序重排所有块的 id：首块 = ``{law_name}-0001``，多 part 共用 base 序号。
    插入类编辑（add）之后调用，保证「id 序号 == 该块在原文中的位置」；
    不改变块内容，只改 id（15 字段其余值不动，part 后缀也保留）。
    """
    out: list[dict] = []
    seq = 0
```

```
    part_base = None
    prev_total = 0
    prev_part_index = 0
    for chunk in chunks:
        item = _copy(chunk)
        try:
            total = int(item.get("part_total") or 1)
            part_index = int(item.get("part_index") or 1)
        except (TypeError, ValueError):
            total, part_index = 1, 1
        if total > 1:
            if part_base is not None and prev_total > 1 and part_index == prev_part_index + 1:
                base = part_base
            else:
                seq += 1
                base = seq
                part_base = base
            item["id"] = f"{law_name}-{base:04d}-p{part_index}"
        else:
            seq += 1
            item["id"] = f"{law_name}-{seq:04d}"
            part_base = None
        prev_total = total
        prev_part_index = part_index
        out.append(item)
    return out
# ------------------------------------------------------------------------
# 各命令
# ------------------------------------------------------------------------
def sort_by_span(chunks: list[dict]) -> list[dict]:
    """按原文行区间排序（稳定）；没有 span 的手工块排在最后，保持相对顺序。"""
    indexed = list(enumerate(chunks or []))
    def key(pair):
        index, chunk = pair
        span = chunk.get("span")
        if isinstance(span, (list, tuple)) and len(span) == 2:
            try:
                return (0, int(span[0]), int(span[1]), index)
            except (TypeError, ValueError):
                pass
        return (1, 0, 0, index)
    return [chunk for _, chunk in sorted(indexed, key=key)]
def normalize_chunks(chunks: list[dict], law_name: str) -> list[dict]:
    """整理成「原文顺序 + id 序号按位置」的规范形态。
    编辑操作可能把新块插到末尾（或插到不相邻的位置），导致列表顺序与原文
    顺序不一致；这里统一排序并按位置重排 id，保证 id 序号始终等于原文位置。
    重排是确定性的，命令重放/撤销时每一步都执行同样的整理，因此 id 一致。
    """
    return _renumber(sort_by_span(chunks), law_name)
def _split(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None) -> l
```

```
ist[dict]:
    index = find_index(chunks, op.get("id"))
    original = chunks[index]
    text = str(original.get("chunk") or "")
    lines = text.split("\n")
    at_line = op.get("at_line")
    at_char = op.get("at_char")
    if at_line is not None:
        try:
            at_line = int(at_line)
        except (TypeError, ValueError):
            raise OpError("at_line 必须是整数") from None
        if not (1 <= at_line < len(lines)):
            raise OpError(f"切分行号越界：{at_line}（该块共 {len(lines)} 行，可取 1–{len(lines
) - 1}）")
        first_text = "\n".join(lines[:at_line])
        second_text = "\n".join(lines[at_line:])
    elif at_char is not None:
        try:
            at_char = int(at_char)
        except (TypeError, ValueError):
            raise OpError("at_char 必须是整数") from None
        if not (1 <= at_char < len(text)):
            raise OpError(f"切分位置越界：{at_char}（该块正文 {len(text)} 字符，可取 1–{len(te
xt) - 1}）")
        first_text, second_text = text[:at_char], text[at_char:]
    else:
        raise OpError("拆分需要 at_line（行级）或 at_char（字符级）")
    first = _copy(original)
    _derive(first, text=first_text)
    _require_text(first, "拆分")
    first["align"] = "manual"
    first["manual"] = True
    second = _copy(original)
    second["id"] = next_id(chunks, law_name, reserved)
    second_law_num = str(op.get("second_law_num") or "").strip()
    if second_law_num:
        second["law_num"] = second_law_num
    second["part_index"] = 1
    second["part_total"] = 1
    second["extras"] = {k: v for k, v in (original.get("extras") or {}).items() if k in INHERI
TED_EXTRAS}
    second["span"] = None
    second["align"] = "manual"
    second["manual"] = True
    _derive(second, text=second_text)
    _require_text(second, "拆分")
    # 给两块补行区间：在「原块 span」窗口内按规范化行顺序定位，
    # 这样拆分后右栏仍显示原文行号、左侧范围条也能立刻断成两段。
    # （只有服务端传入 source_lines 时才做；纯单测调用保持 span=None 的旧行为。）
```

```
    second_span = None
    if source_lines:
        norm = [norm_line(line) for line in source_lines]
        span = original.get("span")
        if span and len(span) == 2:
            lo, hi = max(0, int(span[0]) - 1), min(len(norm), int(span[1]))
        else:
            lo, hi = 0, len(norm)
        first_span = locate_text_span(norm, first_text, lo=lo, hi=hi)
        if first_span:
            first["span"] = [first_span[0] + 1, first_span[1]]
            first["align"] = "manual"
        start_second = first_span[1] if first_span else lo
        second_span = locate_text_span(norm, second_text, lo=start_second, hi=hi)
        if second_span is None:
            second_span = locate_text_span(norm, second_text, lo=lo, hi=hi)
        if second_span:
            second["span"] = [second_span[0] + 1, second_span[1]]
            second["align"] = "manual"
    # 下半块条号：显式填写 > 从下半块正文首行识别 > 从切点所在原文行识别 > 继承原块。
    # 典型场景：两条合并成一块（law_num 保留第一条）后再拆开，下半块应识别出「第二条」。
    if not second_law_num:
        detected = detect_law_num(second_text)
        if not detected and second_span:
            detected = detect_law_num(source_lines[second_span[0]])
        if detected:
            second["law_num"] = detected
    return chunks[:index] + [first, second] + chunks[index + 1 :]
def _merge(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None) -> l
ist[dict]:
    first_id = op.get("id")
    second_id = op.get("with")
    index = find_index(chunks, first_id)
    other = find_index(chunks, second_id)
    if abs(other - index) != 1:
        raise OpError("只能合并相邻的两块（前后紧挨着的两块）")
    # 允许两种写法：{id: 前块, with: 后块} 或 {id: 后块, with: 前块}（前端「并↑/并↓」两种方向
)
    index, other = min(index, other), max(index, other)
    first = chunks[index]
    second = chunks[other]
    merged = _copy(first)
    _derive(merged, text=str(first.get("chunk") or "") + "\n" + str(second.get("chunk") or "")
)
    _require_text(merged, "合并")
    if not str(merged.get("law_num") or "").strip() and str(second.get("law_num") or "").strip
():
        merged["law_num"] = second["law_num"]
    if not (merged.get("extras") or {}) and (second.get("extras") or {}):
        merged["extras"] = copy.deepcopy(second["extras"])
```

```
    span_a, span_b = first.get("span"), second.get("span")
    merged["manual"] = bool(first.get("manual") or second.get("manual"))
    if span_a and span_b:
        # 即便某块是手工改过的（manual），只要两边都有行区间，合并后仍能推出覆盖区间，
        # 这样「段落 + 邻近表格」合并后不会退化成「未对齐」。
        merged["span"] = [min(int(span_a[0]), int(span_b[0])), max(int(span_a[1]), int(span_b[
1]))]
        merged["align"] = "approx"
    else:
        merged["span"] = None
        merged["align"] = "manual"
        merged["manual"] = True
    return chunks[:index] + [merged] + chunks[other + 1 :]
def _merge_many(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None)
 -> list[dict]:
    """把选中的连续多块合并成一块（保留第一块的 id / 条号 / kind）。"""
    ids = op.get("ids")
    if not isinstance(ids, (list, tuple)) or len(ids) < 2:
        raise OpError("集体合并至少需要 2 个切片 id（ids: [\"...\", \"...\"]）")
    indexes = sorted({find_index(chunks, chunk_id) for chunk_id in ids})
    if len(indexes) != len(ids):
        raise OpError("集体合并的 id 列表里有重复项")
    start, end = indexes[0], indexes[-1]
    if indexes != list(range(start, end + 1)):
        raise OpError("集体合并只能选连续的切片（选中的块之间不能跳块）")
    group = chunks[start : end + 1]
    merged = _copy(group[0])
    _derive(merged, text="\n".join(str(chunk.get("chunk") or "") for chunk in group))
    _require_text(merged, "合并")
    if not str(merged.get("law_num") or "").strip():
        for chunk in group[1:]:
            if str(chunk.get("law_num") or "").strip():
                merged["law_num"] = chunk["law_num"]
                break
    if not (merged.get("extras") or {}):
        for chunk in group[1:]:
            if chunk.get("extras"):
                merged["extras"] = copy.deepcopy(chunk["extras"])
                break
    spans = [chunk.get("span") for chunk in group]
    merged["manual"] = any(bool(chunk.get("manual")) for chunk in group)
    if all(span for span in spans):
        merged["span"] = [min(int(span[0]) for span in spans), max(int(span[1]) for span in sp
ans)]
        merged["align"] = "approx"
    else:
        merged["span"] = None
        merged["align"] = "manual"
        merged["manual"] = True
    return chunks[:start] + [merged] + chunks[end + 1 :]
```

```
def _extend(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None) ->
list[dict]:
    """把选区覆盖的连续块并入目标块（保留目标 id），可选覆盖正文/条号。
    形状：``{"op": "extend", "id": 目标, "ids": [目标, …后续块], "chunk": 可选新正文, "law_num
": 可选}``。
    行级语义：调用方（前端）必须把选区吸附到块边界——中间块整块并入，不做半块切分。
    """
    ids = [str(cid) for cid in (op.get("ids") or []) if str(cid or "").strip()]
    target_id = str(op.get("id") or "").strip()
    if not target_id:
        raise OpError("extend 需要 id（目标块）")
    if not ids:
        ids = [target_id]
    if ids[0] != target_id:
        raise OpError("extend 的 ids[0] 必须是目标块 id")
    if len(set(ids)) != len(ids):
        raise OpError("extend 的 ids 里有重复项")
    indexes = [find_index(chunks, cid) for cid in ids]
    if indexes != list(range(indexes[0], indexes[0] + len(indexes))):
        raise OpError("extend 只能合并连续的切片（选中的块之间不能跳块）")
    group = chunks[indexes[0] : indexes[-1] + 1]
    merged = _copy(group[0])
    if "chunk" in op:
        text = str(op.get("chunk") or "")
    else:
        text = "\n".join(str(chunk.get("chunk") or "") for chunk in group)
    _derive(merged, text=text)
    _require_text(merged, "扩展")
    if "law_num" in op:
        law_num = str(op.get("law_num") or "").strip()
        if not law_num:
            raise OpError("law_num 不能为空")
        merged["law_num"] = law_num
    elif not str(merged.get("law_num") or "").strip():
        for chunk in group[1:]:
            if str(chunk.get("law_num") or "").strip():
                merged["law_num"] = chunk["law_num"]
                break
    span = _parse_span(op.get("span"))
    spans = [chunk.get("span") for chunk in group]
    merged["manual"] = True
    merged["align"] = "manual"
    if span:
        merged["span"] = span
    elif all(item and len(item) == 2 for item in spans):
        merged["span"] = [min(int(item[0]) for item in spans), max(int(item[1]) for item in sp
ans)]
    else:
        merged["span"] = None
    out = chunks[: indexes[0]] + [merged] + chunks[indexes[-1] + 1 :]
```

```
    return _renumber(out, law_name)
def _reindex_parts(chunks: list[dict]) -> list[dict]:
    """删除单个 part 后重排同组剩余 part：编号连续、只剩一个时变回普通块。"""
    out = list(chunks)
    i = 0
    while i < len(out):
        if int(out[i].get("part_total") or 1) <= 1:
            i += 1
            continue
        base = RE_PART_SUFFIX.sub("", str(out[i].get("id") or ""))
        group: list[int] = []
        j = i
        while j < len(out) and RE_PART_SUFFIX.sub("", str(out[j].get("id") or "")) == base:
            group.append(j)
            j += 1
        total = len(group)
        for number, idx in enumerate(group, start=1):
            chunk = out[idx]
            if total == 1:
                chunk["id"] = base
                chunk["part_index"] = 1
                chunk["part_total"] = 1
            else:
                chunk["id"] = f"{base}-p{number}"
                chunk["part_index"] = number
                chunk["part_total"] = total
        i = j
    return out
def _drop(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None) -> li
st[dict]:
    index = find_index(chunks, op.get("id"))
    out = chunks[:index] + chunks[index + 1 :]
    if int(chunks[index].get("part_total") or 1) > 1:
        out = _reindex_parts(out)
    return out
def _drop_many(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None)
-> list[dict]:
    """一次删除多个切片（原子命令，一次 Ctrl+Z 撤销）。"""
    ids = op.get("ids")
    if not isinstance(ids, (list, tuple)) or not ids:
        raise OpError("drop_many 需要 ids（非空数组）")
    wanted = [str(item) for item in ids]
    known = {str(chunk.get("id")) for chunk in chunks}
    missing = [item for item in wanted if item not in known]
    if missing:
        raise OpNotFound("找不到切片：" + "、".join(missing[:3]) + ("…" if len(missing) > 3 el
se ""))
    drop = set(wanted)
    out = [chunk for chunk in chunks if str(chunk.get("id")) not in drop]
    return _reindex_parts(out)
```

```
def _clause_anchor(chunk: dict) -> str:
    """取一块可作为条号参照的 law_num（「第X条」或「5.4.1」）；占位/表号/附录号不算。"""
    candidate = str(chunk.get("law_num") or "").strip()
    if not candidate or candidate == NEW_LAW_NUM:
        return ""
    compact = re.sub(r"\s+", "", candidate)
    if not RE_ANCHOR.match(compact):
        return ""
    # 「A.1 表D.1 第1-2行」这类含表号/附录号的复合值不作参照（A.1 开头已被排除，这里防御性检查
)
    if "表" in compact or "附录" in compact:
        return ""
    return compact
def infer_law_num(chunks: list[dict], index: int) -> str:
    """按前后邻居倒推插在第 ``index`` 个位置的新块条号（无绝对把握时返回空串）。
    规则（保守）：
    1. 先看**前一块**（同一块的续写，插在它后面默认仍属于它的条号）；
    2. 前面没有参照时看**后一块**（插在开头，通常属于下一块所在的条）；
    3. 参照只认「第X条」，表号 / 附录号 / （新增）都不拿来推断。
    """
    for j in range(min(index, len(chunks) - 1), -1, -1):
        anchor = _clause_anchor(chunks[j])
        if anchor:
            return anchor
    for j in range(max(0, index + 1), len(chunks)):
        anchor = _clause_anchor(chunks[j])
        if anchor:
            return anchor
    return ""
def _add(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None) -> lis
t[dict]:
    text = str(op.get("text") or "")
    kind = str(op.get("chunk_kind") or "paragraph").strip() or "paragraph"
    if kind not in CHUNK_KINDS:
        raise OpError(f"chunk_kind 非法：{kind!r}（可选 {sorted(CHUNK_KINDS)}）")
    # 插入位置：after_id 之后 / before_id 之前；都不传则追加到末尾
    after_id = op.get("after_id")
    before_id = op.get("before_id")
    if after_id is not None:
        insert_at = find_index(chunks, after_id) + 1
    elif before_id is not None:
        insert_at = find_index(chunks, before_id)
    else:
        insert_at = len(chunks)
    law_num = str(op.get("law_num") or "").strip()
    if not law_num:
        law_num = infer_law_num(chunks, insert_at - 1) or NEW_LAW_NUM
    # 可选：来自原文选区时带上行区间（1-based 闭区间），新增块因此可定位回原文
    span = _parse_span(op.get("span"))
    template = chunks[0] if chunks else {}
```

```
    new: dict = {
        "id": next_id(chunks, law_name, reserved),
        "chunk": "",
        "law_name": law_name,
        "law_num": law_num,
        "law_invalid": template.get("law_invalid") or DEFAULT_LAW_INVALID,
        "law_type": template.get("law_type") or "",
        "law_level": template.get("law_level") or "",
        "issuer": template.get("issuer") or "",
        "effective_date": template.get("effective_date") or "",
        "chunk_kind": kind,
        "token_len": 0,
        "source": template.get("source") or "",
        "part_index": 1,
        "part_total": 1,
        "chunk_hash": "",
        "extras": {"region": "body"},
        "span": span,
        "align": "manual",
        "manual": True,
    }
    _derive(new, text=text)
    _require_text(new, "新增")
    # 新增后按当前顺序重排 id：新块 id 反映它在原文中的位置（插入点之后的块序号顺延），
    # 而不是永远拿队尾的 max_seq+1。
    return _renumber(chunks[:insert_at] + [new] + chunks[insert_at:], law_name)
def _patch(chunks: list[dict], op: dict, law_name: str, reserved=None, source_lines=None) -> l
ist[dict]:
    index = find_index(chunks, op.get("id"))
    chunk = _copy(chunks[index])
    changed_text = False
    if "chunk" in op:
        chunk["chunk"] = str(op.get("chunk") or "")
        changed_text = True
    if "law_num" in op:
        law_num = str(op.get("law_num") or "").strip()
        if not law_num:
            raise OpError("law_num 不能为空")
        chunk["law_num"] = law_num
    if "chunk_kind" in op:
        kind = str(op.get("chunk_kind") or "").strip()
        if kind not in CHUNK_KINDS:
            raise OpError(f"chunk_kind 非法：{kind!r}（可选 {sorted(CHUNK_KINDS)}）")
        chunk["chunk_kind"] = kind
    if "part_index" in op:
        chunk["part_index"] = int(op.get("part_index") or 1)
    if "part_total" in op:
        chunk["part_total"] = int(op.get("part_total") or 1)
    _derive(chunk)
    _require_text(chunk, "修改")
```

```
    if changed_text:
        chunk["manual"] = True
        chunk["align"] = "manual"
    return chunks[:index] + [chunk] + chunks[index + 1 :]
HANDLERS = {
    "split": _split,
    "merge": _merge,
    "merge_many": _merge_many,
    "extend": _extend,
    "drop": _drop,
    "drop_many": _drop_many,
    "add": _add,
    "patch": _patch,
}
# ---------------------------------------------------------------------------
# 对外入口
# ---------------------------------------------------------------------------
def apply_ops(
    chunks: list[dict],
    ops: list[dict],
    *,
    law_name: str,
    source_lines: list[str] | None = None,
    materialize: Callable[[list[dict]], list[dict]] | None = None,
    normalize: Callable[[list[dict]], list[dict]] | None = None,
) -> list[dict]:
    """按顺序应用一串编辑命令，返回新块列表（输入不变）。
    ``reserved`` 记录基线里出现过的全部 id（每步再并入当前结果），
    保证会话内新 id 永不复用已删块编号。
    ``source_lines`` 为源文件行（只读）；传了就为拆分块补行区间，不传则保持旧行为。
    ``materialize`` 是「自动分 part」的物化钩子（由 Studio 传入，避免本模块
    import autopart 造成循环）：每条命令执行后调用一次，把超长块落成真实 part，
    这样后续命令可以直接按 part id 操作、撤销重放也一致。
    ``normalize`` 是「按原文位置排序 + 按位置重排 id」的规范化钩子：启动和
    每条命令后都调用一次，保证列表顺序与 id 序号始终跟原文一致。
    """
    result = [_copy(chunk) for chunk in chunks]
    if materialize is not None:
        result = materialize(result)
    if normalize is not None:
        result = normalize(result)
    reserved = {str(chunk.get("id")) for chunk in (chunks or [])}
    for position, op in enumerate(ops or []):
        if not isinstance(op, dict):
            raise OpError(f"第 {position + 1} 条编辑命令不是 JSON 对象")
        kind = str(op.get("op") or "")
        handler = HANDLERS.get(kind)
        if handler is None:
            raise OpError(f"未知的编辑命令：{kind!r}（可选 {sorted(HANDLERS)}）")
        try:
```

```
            result = handler(result, op, law_name, reserved, source_lines)
        except OpNotFound as exc:
            raise OpNotFound(f"第 {position + 1} 条命令（{kind}）失败：{exc}") from None
        except OpError as exc:
            raise OpError(f"第 {position + 1} 条命令（{kind}）失败：{exc}") from None
        if materialize is not None:
            result = materialize(result)
        if normalize is not None:
            result = normalize(result)
        reserved |= {str(chunk.get("id")) for chunk in result}
    if normalize is not None:
        result = normalize(result)
    return result
def _diff_sig(chunk: dict) -> tuple[str, str, str]:
    """用于 LCS 配对的「块内容签名」：正文 + 条号 + 类型。"""
    return (
        normalize_text(chunk.get("chunk")),
        normalize_text(chunk.get("law_num")),
        str(chunk.get("chunk_kind") or ""),
    )
def _diff_meta(chunk: dict) -> tuple[str, str]:
    """次级配对键：条号 + 类型（正文被改写但条号没动时用）。"""
    return (
        normalize_text(chunk.get("law_num")),
        str(chunk.get("chunk_kind") or ""),
    )
def chunk_diff(baseline: list[dict], current: list[dict]) -> dict[str, Any]:
    """对比基线：新增 / 修改 / 删除的数量与 id 列表。
    因为「新增切片」会按原文位置重排 id（插入点之后的块 id 全部顺延），
    不能再用 id 集合直接比对（会把整段顺延误报成删+增）。这里按内容做
    LCS 配对：内容相同的块视为未动，再按「条号+类型」和「正文相似度」
    把剩余块配成修改，真正多出来的才算新增/删除。
    """
    base = list(baseline or [])
    cur = list(current or [])
    base_used = [False] * len(base)
    cur_used = [False] * len(cur)
    modified: list[str] = []
    matcher = SequenceMatcher(
        a=[_diff_sig(chunk) for chunk in base],
        b=[_diff_sig(chunk) for chunk in cur],
        autojunk=False,
    )
    for tag, i1, i2, j1, j2 in matcher.get_opcodes():
        if tag != "equal":
            continue
        for offset in range(i2 - i1):
            base_used[i1 + offset] = True
            cur_used[j1 + offset] = True
    # 第二阶段：正文改了但条号/类型没动 → 修改配对
```

```
    meta_index: dict[tuple[str, str], list[int]] = {}
    for index in range(len(base)):
        if not base_used[index]:
            meta_index.setdefault(_diff_meta(base[index]), []).append(index)
    rest_cur: list[int] = []
    for index in range(len(cur)):
        if cur_used[index]:
            continue
        bucket = meta_index.get(_diff_meta(cur[index]))
        if bucket:
            base_index = bucket.pop(0)
            base_used[base_index] = True
            cur_used[index] = True
            modified.append(str(cur[index].get("id")))
        else:
            rest_cur.append(index)
    # 第三阶段：条号/类型也改了但正文大体没动 → 按相似度兜底（数量大时跳过，避免卡顿）
    rest_base = [index for index in range(len(base)) if not base_used[index]]
    if rest_base and rest_cur and len(rest_base) * len(rest_cur) <= 2500:
        pairs: list[tuple[float, int, int]] = []
        for base_index in rest_base:
            left = normalize_text(base[base_index].get("chunk"))
            if not left:
                continue
            for cur_index in rest_cur:
                right = normalize_text(cur[cur_index].get("chunk"))
                if not right:
                    continue
                ratio = SequenceMatcher(None, left, right, autojunk=False).ratio()
                if ratio >= 0.5:
                    pairs.append((ratio, base_index, cur_index))
        pairs.sort(reverse=True)
        for _, base_index, cur_index in pairs:
            if base_used[base_index] or cur_used[cur_index]:
                continue
            base_used[base_index] = True
            cur_used[cur_index] = True
            modified.append(str(cur[cur_index].get("id")))
    added = [str(cur[index].get("id")) for index in range(len(cur)) if not cur_used[index]]
    removed = [str(base[index].get("id")) for index in range(len(base)) if not base_used[index
]]
    return {
        "added": len(added),
        "modified": len(modified),
        "removed": len(removed),
        "added_ids": added,
        "modified_ids": modified,
        "removed_ids": removed,
    }
"""自动分 part（P2 编辑后收尾）。
```

```
口径（用户 2026-11-04 决定）
---------------------------
- **文本块**：``token_len > max_chars`` 时均分成 ``-p1..-pN``；行边界优先，
  尾段 < ``min_part_chars`` 时与上一段重排；单行超长才在 ``。；，`` 处硬切；
  每段重复面包屑首行，不做重叠。
- **表格块（只做第一类）**：多行表格块总量超限时按行区间拆成多个独立表格块
  （``表X 第a-b行``）；单行超长的大表格保持原样不拆。
这里是纯函数，不做 IO；由 Studio 在重放编辑命令时调用（``materialize_chunks``
把超长块落成真实块，其它命令可继续按 part 操作）。
"""
from __future__ import annotations
import re
from typing import Any
from .align import norm_line as _norm_line
from .ops import (
    CHUNK_KINDS,
    DEFAULT_LAW_INVALID,
    NEW_LAW_NUM,
    OpError,
    _copy,
    _derive,
    chunk_hash,
    max_seq,
    next_id,
    normalize_text,
)
DEFAULT_MAX_CHARS = 800
DEFAULT_MIN_PART_CHARS = 200
RE_PART_SUFFIX = re.compile(r"-p\d+$")
RE_TABLE_ROWS = re.compile(r"第\s*(\d+)\s*[-—–~～]\s*(\d+)\s*行")
#: 硬切点：句号/分号/逗号（与 FAC-RAG 的 render._hard_split 同口径）。
HARD_CUTS = "。；;，,"
def _is_breadcrumb(line: str) -> bool:
    """块正文首行是不是面包屑（``区域 · 条号`` 这类，且不太长）。"""
    text = str(line or "").strip()
    if not text or len(text) > 160:
        return False
    return " · " in text
def _hard_split_line(line: str, size: int) -> list[str]:
    """把超长单行按标点硬切（切点不早于半段），没有标点就按长度切。"""
    size = max(int(size), 60)
    text = str(line or "")
    pieces: list[str] = []
    while len(text) > size:
        window = text[max(size // 2, 1) : size]
        cut = -1
        for mark in HARD_CUTS:
            pos = window.rfind(mark)
            if pos >= 0:
                cut = max(size // 2, 1) + pos + 1
```

```
                break
        if cut <= 0:
            cut = size
        pieces.append(text[:cut])
        text = text[cut:]
    if text:
        pieces.append(text)
    return pieces
def _pack_lines(lines: list[str], target: int) -> list[str]:
    """按行装箱到 ``target`` 字符左右；单行超长时硬切，切出的片独占一段。"""
    parts: list[str] = []
    current: list[str] = []
    size = 0
    def flush() -> None:
        nonlocal current, size
        if current:
            parts.append("\n".join(current))
            current = []
            size = 0
    for line in lines:
        if len(line) <= target:
            if current and size + 1 + len(line) > target:
                flush()
            current.append(line)
            size += len(line) + 1
            continue
        # 超长行：硬切成片段，每片独占一段（片段之间不拼）
        flush()
        for piece in _hard_split_line(line, target):
            parts.append(piece)
    flush()
    return parts
def _rebalance(parts: list[str], min_chars: int) -> list[str]:
    """尾段过小时从上一段搬行过来（保持行完整，实在搬不动就接受）。"""
    while len(parts) > 1 and len(parts[-1]) < min_chars:
        tail = parts.pop()
        prev_lines = parts.pop().split("\n")
        moved: list[str] = []
        while len("\n".join(moved + [tail])) < min_chars and len(prev_lines) > 1:
            moved.insert(0, prev_lines.pop())
        if not moved:
            # 上一段只有一行（或搬不动任何行）：合并会超长，只能接受现状，
            # 否则这里会一直把同一段 pop 回来造成死循环。
            parts.append("\n".join(prev_lines))
            parts.append(tail)
            break
        tail = "\n".join(moved + [tail])
        if not prev_lines:
            parts.append(tail)
            break
```

```
        parts.append("\n".join(prev_lines))
        parts.append(tail)
    return parts
def split_text_parts(
    text: str,
    max_chars: int = DEFAULT_MAX_CHARS,
    min_chars: int = DEFAULT_MIN_PART_CHARS,
) -> list[str]:
    """把一段正文均分成若干 part（不含 part 编号，仅正文列表）。"""
    body = normalize_text(text)
    if not body:
        return [body]
    total = len(body)
    if total <= max_chars:
        return [body]
    lines = body.split("\n")
    breadcrumb = ""
    if len(lines) > 1 and _is_breadcrumb(lines[0]):
        breadcrumb, lines = lines[0], lines[1:]
    n = max(2, -(-total // max_chars))
    target = max(min_chars, -(-total // n))
    parts = _pack_lines(lines, target)
    parts = _rebalance(parts, min_chars)
    # 目标段数打不住（比如某一行特别长）时，允许段数多于 n，但不再压缩
    if breadcrumb:
        parts = [breadcrumb + "\n" + part if part else breadcrumb for part in parts]
    return parts
def _part_id(base: str, index: int) -> str:
    return f"{RE_PART_SUFFIX.sub('', base)}-p{index}"
def split_text_chunk(
    chunks: list[dict],
    index: int,
    max_chars: int,
    min_chars: int,
) -> list[dict] | None:
    """文本块拆 part；返回新块列表（长度 >=2），不需要拆时返回 None。"""
    chunk = chunks[index]
    parts = split_text_parts(str(chunk.get("chunk") or ""), max_chars, min_chars)
    template = f"{RE_PART_SUFFIX.sub('', str(chunk.get('id') or ''))}"
    if len(parts) < 2:
        return None
    stem = template.rsplit("-", 1)[0] if "-" in template else template
    out: list[dict] = []
    for number, text in enumerate(parts, start=1):
        item = _copy(chunk)
        item["id"] = _part_id(template, number)
        item["part_index"] = number
        item["part_total"] = len(parts)
        if not str(item.get("law_num") or "").strip():
            item["law_num"] = NEW_LAW_NUM
```

```
        _derive(item, text=text)
        item["manual"] = True
        # 继承原块的行区间/对齐状态，让 part 在前端仍能定位到原文
        item["align"] = chunk.get("align") or "manual"
        item["span"] = list(chunk["span"]) if chunk.get("span") else None
        out.append(item)
    # 极端情况下 part id 与现有块碰撞（例如已经手工拆过），顺延到最大序号
    used = {str(c.get("id")) for c in chunks}
    seq = max_seq(chunks)
    for item in out:
        if item["id"] in used and item["id"] != str(chunk.get("id")):
            while True:
                seq += 1
                candidate = f"{stem}-{seq:04d}-p{item['part_index']}"
                if candidate not in used:
                    item["id"] = candidate
                    used.add(candidate)
                    break
    return out
def split_table_chunk(chunks: list[dict], index: int, max_chars: int) -> list[dict] | None:
    """表格块按行区间拆成独立块（只处理多行且超限的表格）。"""
    chunk = chunks[index]
    body = normalize_text(chunk.get("chunk") or "")
    lines = body.split("\n")
    breadcrumb = ""
    if len(lines) > 1 and _is_breadcrumb(lines[0]):
        breadcrumb, rows = lines[0], lines[1:]
    else:
        rows = lines
    if len(rows) < 2:
        return None
    if len(body) <= max_chars:
        return None
    groups: list[list[str]] = []
    current: list[str] = []
    size = len(breadcrumb) + 1 if breadcrumb else 0
    for row in rows:
        if current and size + 1 + len(row) > max_chars:
            groups.append(current)
            current = []
            size = len(breadcrumb) + 1 if breadcrumb else 0
        current.append(row)
        size += len(row) + 1
    if current:
        groups.append(current)
    if len(groups) < 2:
        return None
    law_num = str(chunk.get("law_num") or "")
    match = RE_TABLE_ROWS.search(law_num)
    first_row = int(match.group(1)) if match else None
```

```
    seq = max_seq(chunks)
    out: list[dict] = []
    offset = 0
    for number, group in enumerate(groups):
        item = _copy(chunk)
        if number:
            seq += 1
            item["id"] = f"{str(chunk.get('law_name') or 'chunk')}-{seq:04d}"
        text = (breadcrumb + "\n" if breadcrumb else "") + "\n".join(group)
        _derive(item, text=text)
        item["manual"] = True
        item["align"] = "manual"
        # 行拆块仍落在同一张整表行上，继承原块 span，靠 law_num 行区间区分
        item["span"] = list(chunk["span"]) if chunk.get("span") else None
        item["part_index"] = 1
        item["part_total"] = 1
        if first_row is not None:
            start = first_row + offset
            end = start + len(group) - 1
            item["law_num"] = law_num[: match.start()] + f"第{start}-{end}行" + law_num[match.
end() :]
        offset += len(group)
        out.append(item)
    return out
def _annotate_part_spans(
    parts: list[dict],
    parent: dict,
    source_lines: list[str] | None,
) -> None:
    """给拆出来的 part 标出「真实覆盖的源行区间」（``part_span``，视图字段）。
    part 正文里除面包屑首行外的每一行，在父块 span 范围内按 ``norm_line``
    顺序匹配，得到首/末行；匹配不到就回退父块 span（左侧泳道仍能画外层条）。
    """
    span = list(parent["span"]) if parent.get("span") else None
    if not source_lines or not span:
        for item in parts:
            item["part_span"] = list(span) if span else None
        return
    lo = max(0, int(span[0]) - 1)
    hi = min(len(source_lines), int(span[1]))
    window = [_norm_line(line) for line in source_lines[lo:hi]]
    cursor = 0
    for item in parts:
        lines = str(item.get("chunk") or "").split("\n")
        if lines and _is_breadcrumb(lines[0]):
            lines = lines[1:]
        wanted = [_norm_line(line) for line in lines if _norm_line(line)]
        start = end = -1
        k = cursor
        ok = True
```

```
        for want in wanted:
            hit = -1
            for j in range(k, len(window)):
                if window[j] and (window[j] == want or want in window[j]):
                    hit = j
                    break
            if hit < 0:
                ok = False
                break
            if start < 0:
                start = hit
            end = hit
            k = hit + 1
        if not ok or start < 0:
            item["part_span"] = list(span)
            continue
        item["part_span"] = [lo + start + 1, lo + end + 1]
        cursor = end + 1
RE_TABLE_ROW_LINE = re.compile(r"^(编号|序号|栏号|项目|类别)\s*[:： ]")
#: 表格行正文里常见的字段分隔符（合并块的表格段每行都带这几个键）。
_TABLE_ROW_FIELDS = ("违法行为", "法律规定", "处罚依据", "裁量阶次", "适用条件", "处罚标准")
def _is_table_row_line(line: str) -> bool:
    """判断一行是不是「表格行」正文（``编号：1 · 违法行为：…`` 这种）。
    只认一眼能看出是表格行的：以编号/序号/栏号等开头、且带 `` · `` 字段分隔，
    或者一行里同时出现两个以上表格字段名。避免把普通正文误判成表格行。
    """
    text = str(line or "").strip()
    if not text or " · " not in text:
        return False
    if RE_TABLE_ROW_LINE.match(text):
        return True
    return sum(1 for field in _TABLE_ROW_FIELDS if field + "：" in text) >= 2
def _looks_like_heading(line: str) -> bool:
    """合并块开头的「标题/面包屑」行：短、没有句号、不是表格行。"""
    text = str(line or "").strip()
    if not text or len(text) > 160:
        return False
    if "。" in text:
        return False
    return not _is_table_row_line(text)
def _table_origin_list(originals: list[dict] | None) -> list[dict]:
    """按原始顺序取出所有表格块（面包屑、行区间、侧车都从这里找）。"""
    return [c for c in (originals or []) if str(c.get("chunk_kind") or "") == "table"]
def _table_crumb(chunk: dict) -> str:
    """表格块正文第一行就是它自己的面包屑（例如「现场管理类…台账 · 表3」）。"""
    lines = str(chunk.get("chunk") or "").split("\n")
    return lines[0].strip() if lines else ""
def _table_line_index(originals: list[dict] | None) -> dict[str, list[tuple[int, dict]]]:
    """原始表格块索引：正文每一行（去首行面包屑）→ [(顺序, 块)]。
    同一行文本可能出现在多个表格块里（列名、编号行经常重复，例如两张表的
```

```
    第一行都是「编号：类型」），所以这里保留**所有**候选并按原始块顺序排序；
    匹配时用单调游标挑「当前表格块之后最近的那一个」，避免后面的表把前面
    的行抢走（3010 的 表3/表4 互相串块就是这个问题）。
    """
    index: dict[str, list[tuple[int, dict]]] = {}
    for order, chunk in enumerate(_table_origin_list(originals)):
        body_lines = str(chunk.get("chunk") or "").split("\n")
        for line in body_lines[1:]:
            key = _norm_line(line)
            if key:
                index.setdefault(key, []).append((order, chunk))
    return index
def _table_origin_index(originals: list[dict] | None) -> dict[str, dict]:
    """原始（物化前）表格块索引：按正文最后一行的规范文本找块信息。
    合并块里的表格行就是原来某个表格块的正文行；靠它把行号区间/侧车信息
    找回来，拆出来的表格块才能写回 ``表X 第a-b行``。
    """
    index: dict[str, dict] = {}
    for chunk in originals or []:
        if str(chunk.get("chunk_kind") or "") != "table":
            continue
        body_lines = str(chunk.get("chunk") or "").split("\n")
        if not body_lines:
            continue
        key = _norm_line(body_lines[-1])
        if key:
            index.setdefault(key, chunk)
    return index
def split_mixed_chunk(
    chunks: list[dict],
    index: int,
    max_chars: int,
    min_chars: int,
    originals: list[dict] | None = None,
    source_lines: list[str] | None = None,
    grids: dict[str, dict] | None = None,
) -> list[dict] | None:
    """把「文本 + 表格」混合的超长块按类型边界拆开（用户 m10872 定的方案）。
    - 文本段 → 普通段落块（过长再均分 part）；
    - 表格段 → 每个表格行一个独立表格块，行号区间/表格侧车从原始表格块找回；
    - 面包屑只在第一块出现；
    - 不是混合块（只有一种类型）时返回 None，交给原来的拆分逻辑。
    """
    chunk = chunks[index]
    body = normalize_text(chunk.get("chunk") or "")
    if len(body) <= max_chars:
        return None
    lines = [line for line in body.split("\n") if line.strip()]
    if not lines:
        return None
```

```
          selectByKey(key);
        } else {
          setStatus("已重新扫描 · " + res.files.totals.n_files + " 个文件 / " + res.files.tota
ls.n_chunks + " 块", "ok");
        }
        showToast("已重新扫描数据目录：" + res.files.totals.n_files + " 个文件 · " + res.files
.totals.n_chunks + " 块", "ok");
      }).catch(function (err) {
        showToast("重新扫描失败：" + err.message, "err");
        loadFiles();
      }).finally(function () {
        btn.disabled = false;
        btn.textContent = label;
      });
    });
    el.btnSettings.addEventListener("click", openSettings);
    if (el.btnChunk) {
      el.btnChunk.addEventListener("click", function (ev) {
        ev.preventDefault();
        triggerChunk();
      });
    }
    el.btnCancel.addEventListener("click", function () { el.dialog.close(); });
    el.form.addEventListener("submit", saveSettings);
    var laneToggle = document.getElementById("toggle-lanes");
    if (laneToggle) {
      var stored = null;
      try { stored = window.localStorage.getItem("sca.lanes"); } catch (e) { stored = null; }
      laneToggle.checked = stored !== "0";
      applyLanesVisibility(laneToggle.checked);
      laneToggle.addEventListener("change", function () {
        applyLanesVisibility(laneToggle.checked);
        try { window.localStorage.setItem("sca.lanes", laneToggle.checked ? "1" : "0"); } catc
h (e) { /* 忽略 */ }
      });
    }
    // P2 编辑：按钮 + 弹层
    el.btnUndo.addEventListener("click", doUndo);
    el.btnRedo.addEventListener("click", doRedo);
    el.btnReset.addEventListener("click", doReset);
    if (el.btnSaveDraft) el.btnSaveDraft.addEventListener("click", function () { saveDraft();
});
    if (el.btnCommit) el.btnCommit.addEventListener("click", commitDraftAction);
    if (el.btnDiscard) el.btnDiscard.addEventListener("click", discardDraftAction);
    if (el.btnExport) el.btnExport.addEventListener("click", exportChunks);
    el.btnAdd.addEventListener("click", function (ev) {
    ev.preventDefault();
    var picked = selectedTableRows();
    openAdd(picked || undefined);
  });
```

```
    // 点「新增」按钮时不抢焦点/不清掉原文选区
    el.btnAdd.addEventListener("mousedown", function (ev) { ev.preventDefault(); });
    el.splitForm.addEventListener("submit", submitSplit);
    el.splitLines.addEventListener("change", updateSplitHint);
    el.splitChar.addEventListener("input", updateSplitHint);
    el.editForm.addEventListener("submit", submitEdit);
    el.addForm.addEventListener("submit", submitAdd);
    Array.prototype.forEach.call(document.querySelectorAll(".btn-cancel-op"), function (btn) {
      btn.addEventListener("click", function () {
        [el.splitDialog, el.editDialog, el.addDialog].forEach(function (dlg) {
          if (dlg && dlg.open) dlg.close();
        });
      });
    });
    document.addEventListener("keydown", function (ev) {
      var tag = (ev.target && ev.target.tagName) || "";
      var typing = tag === "INPUT" || tag === "TEXTAREA" || tag === "SELECT" ||
        !!(ev.target && ev.target.isContentEditable);
      var mod = ev.ctrlKey || ev.metaKey;
      var key = String(ev.key || "").toLowerCase();
      if (ev.key === "Escape") {
        if (state.deleteMode) setDeleteMode(false);
        hideCtxMenu();
        hideSplitCaret();
        hideRowPick();
        clearMultiSel();
        state.suppressNextClick = false;
        [el.dialog, el.splitDialog, el.editDialog, el.addDialog, el.choiceDialog].forEach(func
tion (dlg) {
          if (dlg && dlg.open) dlg.close();
        });
        return;
      }
      if (mod && key === "k") {
        ev.preventDefault();
        el.search.focus();
        return;
      }
      if (mod && key === "b") {
        ev.preventDefault();
        toggleSidebar();
        return;
      }
      if (mod && key === "s") {
        ev.preventDefault();
        if (state.doc && state.doc.dirty) saveDraft();
        return;
      }
      if (typing) return;
      if (mod && !ev.shiftKey && key === "z") {
```

```
        ev.preventDefault();
        doUndo();
        return;
      }
      if (mod && (key === "y" || (ev.shiftKey && key === "z"))) {
        ev.preventDefault();
        doRedo();
        return;
      }
      if (mod && ev.shiftKey && key === "n") {
        ev.preventDefault();
        openAdd();
        return;
      }
      if (mod && key === "enter" && state.activeChunk) {
        ev.preventDefault();
        openSplit(findChunk(state.activeChunk));
        return;
      }
      if (key === "delete") {
        var multi = selectedMultiChunks();
        if (multi.length) {
          ev.preventDefault();
          dropChunks(multi);
          clearMultiSel();
          return;
        }
        if (state.activeChunk) {
          ev.preventDefault();
          requestDelete(findChunk(state.activeChunk));
        }
      }
    });
    bindDocEvents();
    // 点菜单外部 / 滚动原文或切片 / 窗口尺寸变化时收起右键菜单与拆分光标
    document.addEventListener("click", function (ev) {
      var inMenu = el.ctxMenu.contains(ev.target);
      var inTable = ev.target && ev.target.closest && ev.target.closest(".line.table-block");
      if (!el.ctxMenu.hidden && !inMenu) hideCtxMenu();
      if (el.source && !el.source.contains(ev.target)) hideSplitCaret();
      // 行选区只属于当前表格：点到别处（菜单/对话框/按钮除外）即收起
      var keepPick = ev.target && ev.target.closest && ev.target.closest("button, dialog, #chu
nks, .toolbar");
      if (!inMenu && !inTable && !keepPick && state.rowPick) hideRowPick();
    });
    el.source.addEventListener("scroll", function () {
      if (!programmaticScrollNow()) hideCtxMenu();
      hideSplitCaret();
    });
    el.chunks.addEventListener("scroll", function () {
```

```
      if (!programmaticScrollNow()) hideCtxMenu();
    });
    window.addEventListener("resize", function () { hideCtxMenu(); hideSplitCaret(); });
    window.addEventListener("hashchange", function () {
      var key = keyFromHash();
      if (key && key !== state.docKey) selectByKey(key);
    });
    bindUploadEvents();
  }
  bindEvents();
  bindLayout();
  initHealth();
  loadFiles();
  pollWatch();
})();
/* SplitChunkAssistant 样式（零依赖，手写 CSS）—— P1 三栏：文件树 / 原文 / 切片 */
:root {
  --bg: #f6f7f9;
  --panel: #ffffff;
  --border: #d7dbe2;
  --text: #1f2430;
  --muted: #6b7280;
  --accent: #2563eb;
  --accent-weak: #dbe6ff;
  --ok: #15803d;
  --warn: #b45309;
  --err: #b91c1c;
  --radius: 8px;
  --mono: "Cascadia Mono", "Consolas", "Source Code Pro", monospace;
}
* { box-sizing: border-box; }
/* hidden 属性优先级最高：避免被后面的 display:flex / inline-flex 覆盖（删除模式浮条等） */
[hidden] { display: none !important; }
html, body {
  margin: 0;
  height: 100%;
  background: var(--bg);
  color: var(--text);
  font: 14px/1.55 "Segoe UI", "Microsoft YaHei", system-ui, sans-serif;
}
body {
  display: flex;
  flex-direction: column;
  height: 100vh;
  overflow: hidden;
}
/* ---------- 顶栏 ---------- */
.topbar {
  display: flex;
  align-items: center;
```

```
  gap: 14px;
  padding: 10px 16px;
  background: var(--panel);
  border-bottom: 1px solid var(--border);
  flex: none;
  z-index: 10;
}
.brand {
  display: flex;
  align-items: center;
  gap: 10px;
  min-width: 260px;
}
.logo {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 28px;
  height: 28px;
  border-radius: 8px;
  background: var(--accent);
  color: #fff;
  font-weight: 700;
  font-size: 12px;
  letter-spacing: .5px;
}
.brand h1 { font-size: 16px; margin: 0; font-weight: 650; }
.phase {
  font-size: 11px;
  color: var(--accent);
  background: var(--accent-weak);
  border-radius: 999px;
  padding: 1px 8px;
}
.status {
  flex: 1;
  color: var(--muted);
  font-size: 13px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.status.ok { color: var(--ok); }
.status.warn { color: var(--warn); }
.status.err { color: var(--err); font-weight: 600; }
.actions { display: flex; gap: 8px; }
/* ---------- 按钮 ---------- */
.btn {
  border: 1px solid var(--border);
  background: var(--panel);
```

```
  color: var(--text);
  border-radius: var(--radius);
  padding: 6px 12px;
  font-size: 13px;
  cursor: pointer;
  transition: background .15s, border-color .15s;
}
.btn:hover { background: #eef1f6; border-color: #b9c0ca; }
.btn:active { background: #e2e7ef; }
.btn-primary { background: var(--accent); border-color: var(--accent); color: #fff; }
.btn-primary:hover { background: #1d4ed8; border-color: #1d4ed8; }
/* ---------- 三栏布局 ---------- */
.layout {
  --sidebar-w: 300px;
  --chunks-w: 420px;
  display: grid;
  grid-template-columns: var(--sidebar-w) minmax(0, 1fr) minmax(300px, var(--chunks-w));
  flex: 1;
  min-height: 0;
  overflow: hidden;
}
/* 折叠左栏：空出的一列交给原文栏（1fr 自动填充），切片栏宽度保持用户设定 */
.layout.sidebar-collapsed { grid-template-columns: 0 minmax(0, 1fr) minmax(300px, var(--chunks
-w)); }
.layout.sidebar-collapsed .sidebar { display: none; }
/* 左栏 display:none 后自动排布会把原文挤到第一列，这里显式钉回第 2/3 列 */
.layout.sidebar-collapsed .source-pane { grid-column: 2; }
.layout.sidebar-collapsed .chunks-pane { grid-column: 3; }
/* 左：文件树 */
.sidebar {
  display: flex;
  flex-direction: column;
  background: var(--panel);
  border-right: 1px solid var(--border);
  min-height: 0;
  padding: 12px;
  gap: 10px;
}
.search-row input {
  width: 100%;
  padding: 8px 10px;
  border: 1px solid var(--border);
  border-radius: var(--radius);
  font-size: 13px;
  outline: none;
  background: #fbfcfe;
}
.search-row input:focus { border-color: var(--accent); box-shadow: 0 0 0 3px var(--accent-weak
); }
.stats { color: var(--muted); font-size: 12px; line-height: 1.6; padding: 0 2px; }
```

```
.tree {
  overflow-y: auto;
  flex: 1;
  min-height: 0;
  border-top: 1px solid var(--border);
  padding-top: 8px;
}
.group { margin-bottom: 10px; }
.group-head {
  display: flex;
  justify-content: space-between;
  align-items: baseline;
  padding: 4px 6px;
  font-size: 12px;
  color: var(--muted);
  font-weight: 600;
}
.group-head .count { font-weight: 400; }
.file {
  display: block;
  width: 100%;
  text-align: left;
  border: 0;
  background: transparent;
  border-radius: 6px;
  padding: 6px 26px 6px 8px;
  cursor: pointer;
  font: inherit;
  color: var(--text);
  position: relative;
}
.file:hover { background: #eef2f9; }
.file.active { background: var(--accent-weak); outline: 1px solid #b9cdf6; }
.file .name {
  display: block;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  font-size: 13px;
}
.file .meta { display: flex; gap: 8px; font-size: 11px; color: var(--muted); margin-top: 1px;
}
/* 审核状态点：固定在文件卡右上角 */
.file .review-dot { position: absolute; top: 10px; right: 10px; margin: 0; }
.badge {
  border-radius: 999px;
  padding: 0 6px;
  font-size: 10.5px;
  line-height: 16px;
  border: 1px solid transparent;
```

```
  white-space: nowrap;
}
.badge-ok { color: var(--ok); background: #e7f6ec; }
.badge-warn { color: var(--warn); background: #fdf1e3; }
.badge-err { color: var(--err); background: #fdeaea; }
.badge-local { color: #0f766e; background: #e8f7f3; border-color: #b7e4d8; }
.badge-tblhint { color: #92400e; background: #fef3c7; border-color: #fcd34d; }
.empty-hint { color: var(--muted); font-size: 13px; padding: 18px 8px; text-align: center; }
/* 中：原文 / 右：切片 */
.source-pane,
.chunks-pane {
  display: flex;
  flex-direction: column;
  min-height: 0;
  min-width: 0;
  background: var(--panel);
}
.chunks-pane { border-left: 1px solid var(--border); position: relative; }
/* 原文 / 切片之间的拖拽条 */
.resizer {
  position: absolute; left: -4px; top: 0; bottom: 0; width: 8px;
  cursor: col-resize; z-index: 6; background: transparent; touch-action: none;
}
.resizer:hover, .resizer.dragging {
  background: rgba(37, 99, 235, .16);
  box-shadow: inset 0 0 0 1px rgba(37, 99, 235, .35);
}
body.resizing, body.resizing * { cursor: col-resize !important; user-select: none !important;
}
.btn-icon { padding: 2px 9px; font-size: 14px; line-height: 18px; }
.pane-head {
  flex: none;
  padding: 8px 12px;
  border-bottom: 1px solid var(--border);
  background: #fbfcfe;
}
.pane-title {
  font-size: 13px;
  font-weight: 650;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.pane-title .count { font-weight: 400; color: var(--muted); font-size: 12px; }
.pane-meta {
  margin-top: 1px;
  font-size: 11.5px;
  color: var(--muted);
  overflow: hidden;
  text-overflow: ellipsis;
```

```
  white-space: nowrap;
}
.source, .chunks {
  flex: 1;
  min-height: 0;
  overflow: auto;
  padding: 10px 12px;
}
/* 原文行 */
.source { font-family: var(--mono); font-size: 12.5px; }
.line {
  display: grid;
  grid-template-columns: 52px 20px minmax(0, 1fr);
  padding: 0 4px;
  border-radius: 4px;
  line-height: 1.55;
  transition: background .08s;
}
.line .no {
  color: #9aa3b2;
  text-align: right;
  padding-right: 10px;
  font-size: 11px;
  user-select: none;
}
/* 切片范围条（泳道）：每块的行区间画成一根竖条，4 条泳道承载重叠 */
.line .lanes { position: relative; user-select: none; }
.line .lane {
  position: absolute;
  top: 0;
  bottom: 0;
  width: 3px;
  border-radius: 1px;
  opacity: .8;
  cursor: pointer;
}
.line .lane:hover { opacity: 1; }
.line .lane.start { top: 3px; border-top-left-radius: 2px; border-top-right-radius: 2px; }
.line .lane.end { bottom: 3px; border-bottom-left-radius: 2px; border-bottom-right-radius: 2px
; }
.line .lane.only { top: 3px; bottom: 3px; }
.line .lane-0 { left: 2px; background: #60a5fa; }
.line .lane-1 { left: 6px; background: #34d399; }
.line .lane-2 { left: 10px; background: #f59e0b; }
.line .lane-3 { left: 14px; background: #a78bfa; }
/* P3 分片泳道：外层浅条 = 父块整段；内层深条 = part 真实行段 */
.line .lane.lane-parent { left: 1px; width: 5px; background: #c7ddff; opacity: .95; border-rad
ius: 3px; }
.line .lane.lane-part { left: 7px; width: 3px; background: #1d4ed8; opacity: 1; }
.line .lane.lane-part.start { top: 4px; }
```

```
.line .lane.lane-part.end { bottom: 4px; }
/* 表格块：一张表被切成很多块、span 又都是同一源码行时，合并成一根聚合条 */
.line .lane-agg {
  left: 4px;
  width: 5px;
  border-radius: 3px;
  background: repeating-linear-gradient(
    180deg,
    #60a5fa 0 10px,
    #34d399 10px 20px,
    #f59e0b 20px 30px,
    #a78bfa 30px 40px
  );
}
body.no-lanes .line { grid-template-columns: 52px minmax(0, 1fr); }
body.no-lanes .line .lanes { display: none; }
.lane-toggle {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  float: right;
  margin-top: -18px;
  font-size: 11px;
  color: var(--muted);
  cursor: pointer;
  user-select: none;
}
.lane-toggle input { margin: 0; cursor: pointer; }
.line .txt { white-space: pre-wrap; word-break: break-word; }
.line.blank { min-height: 1.2em; }
.line.kind-h1, .line.kind-h2, .line.kind-h3, .line.kind-h {
  font-weight: 700;
  color: #111827;
}
.line.kind-h1 .txt { font-size: 14.5px; }
.line.kind-h2 .txt { font-size: 13.5px; }
.line.kind-quote .txt { color: #4b5563; border-left: 2px solid #d1d5db; padding-left: 6px; }
.line.kind-table .txt { color: #334155; }
.line.truncated .txt { color: #7c3aed; }
.line.hl { background: #e6efff; box-shadow: inset 3px 0 0 var(--accent); }
.line.hl-soft { background: #eef4ff; }
.line.sel { background: #fff7e0; box-shadow: inset 3px 0 0 #f59e0b; }
/* 切片卡片 */
.chunks { font-size: 13px; }
.chunk-card {
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--panel);
  padding: 8px 10px;
  margin-bottom: 8px;
```

```
  cursor: pointer;
  transition: border-color .12s, box-shadow .12s;
}
.chunk-card:hover { border-color: #9dbcf5; }
.chunk-card.related { border-color: #93b4f5; box-shadow: 0 0 0 2px #e3ecff; }
.chunk-card.hit { border-color: #f59e0b; box-shadow: 0 0 0 2px #fde68a; }
.chunk-card.active { border-color: var(--accent); box-shadow: 0 0 0 2px var(--accent-weak); }
.chunk-card.align-miss { border-style: dashed; }
.chunk-card.align-miss .chunk-body { color: #6b7280; }
.chunk-head {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 4px;
  font-size: 10.5px;
  color: var(--muted);
}
.chunk-seq { font-weight: 700; color: var(--accent); margin-right: 2px; }
.badge-kind { background: #eef1f7; color: #334155; }
.badge-kind.kind-table { background: #e8f7f3; color: #0f766e; }
.badge-kind.kind-clause { background: #eef2ff; color: #3730a3; }
.badge-kind.kind-appendix { background: #faf0ff; color: #7e22ce; }
.badge-kind.kind-paragraph { background: #f2f4f8; color: #475569; }
.badge-kind.kind-stub { background: #fdeaea; color: var(--err); }
.badge-num { background: #fff7e6; color: #92400e; }
.badge-part { background: #f1f5f9; color: #475569; }
.badge-align { border: 1px solid transparent; }
.badge-align.exact { background: #e7f6ec; color: var(--ok); }
.badge-align.substr { background: #e6efff; color: var(--accent); }
.badge-align.table_rows { background: #e8f7f3; color: #0f766e; }
.badge-align.table_label { background: #fdf1e3; color: var(--warn); }
.badge-align.miss { background: #f3f4f6; color: #6b7280; }
.chunk-body {
  margin: 6px 0 4px;
  padding: 6px 8px;
  font-family: var(--mono);
  font-size: 11.5px;
  line-height: 1.55;
  white-space: pre-wrap;
  word-break: break-word;
  background: #fbfcfe;
  border: 1px dashed #e3e7ef;
  border-radius: 6px;
  max-height: 240px;
  overflow: auto;
}
.chunk-card.active .chunk-body { max-height: 520px; background: #fafdff; }
.chunk-foot {
  display: flex;
  justify-content: space-between;
```

```
  gap: 8px;
  color: #9aa3b2;
  font-family: var(--mono);
  font-size: 10.5px;
}
.chunk-foot span { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.chunk-foot .chunk-span { color: var(--muted); flex: none; }
/* 空态 */
.empty {
  max-width: 460px;
  margin: 10vh auto 0;
  text-align: center;
  color: var(--muted);
}
.empty h2 { color: var(--text); font-size: 17px; margin-bottom: 6px; }
.empty p { font-size: 13px; }
/* ---------- 设置对话框 ---------- */
.dialog {
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 0;
  width: min(680px, 92vw);
  box-shadow: 0 18px 60px rgba(15, 23, 42, .22);
}
.dialog::backdrop { background: rgba(15, 23, 42, .35); }
.dialog form { padding: 20px 22px; }
.dialog h2 { margin: 0 0 4px; font-size: 18px; }
.hint { color: var(--muted); font-size: 12.5px; margin: 0 0 16px; }
.field { display: block; margin-bottom: 14px; }
.field > span {
  display: block;
  font-size: 12.5px;
  color: var(--muted);
  margin-bottom: 5px;
}
.field input {
  width: 100%;
  padding: 7px 10px;
  border: 1px solid var(--border);
  border-radius: var(--radius);
  font-size: 13px;
  font-family: var(--mono);
  background: #fbfcfe;
}
.field input:focus { border-color: var(--accent); outline: none; box-shadow: 0 0 0 3px var(--a
ccent-weak); }
/* 设置面板里「勾选 + 说明」同一行 */
.field.inline-field { display: flex; align-items: center; gap: 10px; justify-content: space-be
tween; }
.field.inline-field > span { margin-bottom: 0; flex: 1; }
```

```
.field.inline-field input[type="checkbox"] { width: auto; flex: none; transform: scale(1.15);
cursor: pointer; }
.source-roots { display: grid; gap: 8px; }
.source-row { display: grid; grid-template-columns: 150px 1fr; gap: 8px; align-items: center;
}
.source-row label { font-family: var(--mono); font-size: 12px; color: var(--muted); }
.dialog-errors {
  background: #fdeaea;
  border: 1px solid #f3c2c2;
  color: var(--err);
  border-radius: var(--radius);
  padding: 10px 12px;
  font-size: 12.5px;
  margin-bottom: 14px;
  white-space: pre-wrap;
}
.dialog-actions { display: flex; justify-content: flex-end; gap: 8px; margin-top: 6px; }
/* ---------- Toast ---------- */
.toast {
  position: fixed;
  left: 50%;
  bottom: 26px;
  transform: translateX(-50%);
  background: #1f2430;
  color: #fff;
  padding: 9px 16px;
  border-radius: 999px;
  font-size: 13px;
  box-shadow: 0 8px 30px rgba(0, 0, 0, .25);
  z-index: 100;
}
.toast.err { background: var(--err); }
.toast.ok { background: var(--ok); }
/* ---------- 响应式 ---------- */
@media (max-width: 1180px) {
  .layout { --sidebar-w: 250px; --chunks-w: 380px; }
  .brand { min-width: 0; }
}
@media (max-width: 900px) {
  .layout { --sidebar-w: 220px; grid-template-columns: var(--sidebar-w) minmax(0, 1fr); grid-t
emplate-rows: 1fr 40vh; }
  .layout.sidebar-collapsed { grid-template-columns: 0 minmax(0, 1fr); }
  .resizer { display: none; }
  .chunks-pane { grid-column: 1 / -1; border-left: 0; border-top: 1px solid var(--border); }
  .topbar { flex-wrap: wrap; }
  .status { order: 3; width: 100%; }
}
/* ---------- P2：编辑工具条 / 卡片操作 / 弹层 ---------- */
.btn:disabled { opacity: .45; cursor: not-allowed; }
.btn:disabled:hover { background: var(--panel); border-color: var(--border); }
```

```
.btn-small { padding: 3px 9px; font-size: 11.5px; border-radius: 6px; }
.edit-tools {
  display: flex;
  align-items: center;
  gap: 8px;
  flex: none;
}
.edit-tools .dirty {
  color: var(--err);
  background: #fdeaea;
  border-radius: 999px;
  padding: 1px 9px;
  font-size: 11.5px;
  white-space: nowrap;
}
.draft-badge {
  border-radius: 999px;
  padding: 2px 10px;
  font-size: 11.5px;
  white-space: nowrap;
  border: 1px solid transparent;
}
.draft-badge.dirty { color: #b45309; background: #fef3c7; border-color: #fcd34d; }
.draft-badge.saved { color: #1d4ed8; background: #e8efff; border-color: #bfd3ff; }
.draft-badge.stale { color: #b91c1c; background: #fef2f2; border-color: #fecaca; }
.badge-draft { color: #1d4ed8; background: #e8efff; border-color: #bfd3ff; }
.edit-tools .diff {
  font-family: var(--mono);
  font-size: 11.5px;
  color: var(--muted);
  background: #f2f4f8;
  border-radius: 999px;
  padding: 2px 9px;
  white-space: nowrap;
}
.edit-tools .diff.editing { color: var(--warn); background: #fdf1e3; }
.pane-actions {
  float: right;
  display: flex;
  gap: 6px;
  margin-top: -19px;
}
.chunks-hint {
  flex: none;
  padding: 6px 12px;
  font-size: 11.5px;
  color: #92400e;
  background: #fffbeb;
  border-bottom: 1px solid #fde68a;
}
```

```
.chunk-ops {
  display: flex;
  flex-wrap: wrap;
  gap: 5px;
  margin-top: 7px;
  padding-top: 6px;
  border-top: 1px dashed #e3e7ef;
}
.chip {
  border: 1px solid var(--border);
  background: #fbfcfe;
  color: #475569;
  border-radius: 6px;
  padding: 2px 8px;
  font-size: 11.5px;
  line-height: 1.5;
  cursor: pointer;
  transition: background .12s, border-color .12s;
}
.chip:hover { background: #eef2ff; border-color: #9dbcf5; color: var(--accent); }
.chip:disabled { opacity: .4; cursor: not-allowed; }
.chip:disabled:hover { background: #fbfcfe; border-color: var(--border); color: #475569; }
.badge-manual { background: #fdf1e3; color: var(--warn); }
.badge-cover { background: #eef2ff; color: #4338ca; cursor: help; }
.op-dialog { width: min(640px, 94vw); }
.op-target {
  font-family: var(--mono);
  font-size: 11.5px;
  color: var(--muted);
  background: #f2f4f8;
  border-radius: 6px;
  padding: 6px 10px;
  margin-bottom: 12px;
  word-break: break-all;
}
.op-hint {
  margin: 0 0 12px;
  padding: 7px 10px;
  border-radius: 6px;
  background: #eef6ff;
  border: 1px solid #cfe3ff;
  color: #1d4ed8;
  font-size: 12.5px;
}
.split-lines {
  display: grid;
  gap: 4px;
  max-height: 240px;
  overflow: auto;
  margin-bottom: 12px;
```

```
}
.split-line-opt {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 5px 8px;
  border: 1px solid transparent;
  border-radius: 6px;
  cursor: pointer;
  font-size: 12.5px;
}
.split-line-opt:hover { background: #f5f8ff; border-color: #dbe6ff; }
.split-line-opt input { accent-color: var(--accent); }
.field textarea,
.field select {
  width: 100%;
  padding: 7px 10px;
  border: 1px solid var(--border);
  border-radius: var(--radius);
  font-size: 12.5px;
  font-family: var(--mono);
  background: #fbfcfe;
  color: var(--text);
}
.field textarea { resize: vertical; min-height: 120px; line-height: 1.55; }
.field textarea:focus,
.field select:focus { border-color: var(--accent); outline: none; box-shadow: 0 0 0 3px var(--
accent-weak); }
.btn-cancel-op { background: #fafbfc; }
/* ---------- 右键上下文菜单（P2） ---------- */
.ctx-menu {
  position: fixed;
  z-index: 60;
  min-width: 230px;
  padding: 5px;
  border: 1px solid var(--line);
  border-radius: 10px;
  background: #fff;
  box-shadow: 0 12px 32px rgba(15, 23, 42, 0.18);
  font-size: 13px;
  user-select: none;
}
.ctx-menu[hidden] { display: none; }
.ctx-item {
  display: flex;
  width: 100%;
  align-items: baseline;
  gap: 8px;
  padding: 7px 10px;
  border: none;
```

```
  border-radius: 7px;
  background: transparent;
  color: var(--text);
  font: inherit;
  text-align: left;
  cursor: pointer;
}
.ctx-item:hover:not(:disabled) { background: #eef4ff; }
.ctx-item:disabled { color: var(--muted); cursor: default; opacity: .62; }
.ctx-label { flex: none; white-space: nowrap; }
.ctx-hint {
  flex: 1 1 auto;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  text-align: right;
  color: var(--muted);
  font-size: 11.5px;
  font-family: var(--mono);
}
.ctx-sep { height: 1px; margin: 5px 8px; background: var(--line); }
/* ---------- 光标反馈 & 位置提示（P2） ---------- */
.source-tip {
  padding: 4px 14px 6px;
  color: var(--muted);
  font-size: 12px;
  line-height: 1.55;
  border-bottom: 1px solid var(--line);
  background: #f8fafc;
}
.source-tip b { color: #1d4ed8; font-weight: 600; }
.split-caret {
  position: fixed;
  width: 2px;
  background: #2563eb;
  border-radius: 1px;
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.16);
  pointer-events: none;
  z-index: 55;
  animation: split-caret-blink 1.1s steps(2, start) infinite;
}
@keyframes split-caret-blink { 0%, 60% { opacity: 1; } 61%, 100% { opacity: 0.35; } }
.split-hint {
  position: fixed;
  z-index: 56;
  pointer-events: none;
  max-width: min(560px, 86vw);
  padding: 4px 9px;
  border-radius: 7px;
  background: #1d4ed8;
```

```
  color: #fff;
  font-size: 12px;
  line-height: 1.5;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  box-shadow: 0 6px 16px rgba(15, 23, 42, 0.28);
}
.chunk-card.target {
  border-color: #2563eb;
  box-shadow: 0 0 0 2px rgba(37, 99, 235, 0.35);
}
/* ---------- 表格可视化（管道表 / HTML 表 / table_grid） ---------- */
.line.table-block { align-items: start; }
/* 表格块很高：泳道格子必须拉伸到整块高度，否则范围条（绝对定位竖条）高度为 0 不可见 */
.line.table-block .lanes { align-self: stretch; }
.line.table-block .no { padding-top: 4px; font-size: 10.5px; color: #8a93a2; }
.line.table-block .txt { min-width: 0; overflow: hidden; padding: 2px 2px 4px 0; }
.line.table-cont { display: none; }
.line.table-block.hl { box-shadow: inset 3px 0 0 var(--accent); }
.line.table-block.hl .tblwrap, .line.table-block.hl-soft .tblwrap { border-color: #b9cdf0; }
.line.table-block.hl-soft { background: #eef4ff; }
.tblmeta {
  color: #6b7280;
  font-size: 10.5px;
  margin: 2px 0 3px;
  font-family: var(--mono);
  line-height: 1.4;
}
.tblwrap {
  overflow: auto;
  max-height: 380px;
  border: 1px solid #e2e5ea;
  border-radius: 6px;
  background: #fff;
}
.grid-table {
  border-collapse: collapse;
  font-family: var(--mono);
  font-size: 11.5px;
  line-height: 1.5;
  min-width: 100%;
}
.grid-table th, .grid-table td {
  border: 1px solid #e2e5ea;
  padding: 4px 7px;
  text-align: left;
  vertical-align: top;
  white-space: normal;
  word-break: break-word;
```

```
}
.grid-table thead th {
  background: #eef2f6;
  position: sticky;
  top: 0;
  z-index: 1;
  font-weight: 600;
  white-space: nowrap;
}
.grid-table tbody tr:nth-child(even) td { background: #fafbfc; }
.grid-table tr.noterow td { background: #fff8e8; color: #6b4a00; font-style: italic; }
/* 表格内部按切片着色：每个表格切片覆盖的行区间涂上它自己的颜色，点行选中切片 */
.grid-table tbody tr.chunk-row { cursor: pointer; }
.grid-table tbody tr.chunk-row td {
  background: hsla(var(--row-hue, 210), 70%, 55%, 0.10);
}
.grid-table tbody tr.chunk-row td:first-child {
  border-left: 3px solid hsl(var(--row-hue, 210), 65%, 45%);
}
.grid-table tbody tr.chunk-row:hover td {
  background: hsla(var(--row-hue, 210), 75%, 55%, 0.24);
}
.grid-table tbody tr.chunk-row.row-hl td,
.grid-table tbody tr.chunk-row.row-hl-soft td {
  box-shadow: inset 0 0 0 1px hsl(var(--row-hue, 210), 65%, 45%);
}
.grid-table tbody tr.chunk-row.row-active td {
  background: hsla(var(--row-hue, 210), 80%, 55%, 0.32);
  box-shadow: inset 0 0 0 2px hsl(var(--row-hue, 210), 70%, 42%);
}
/* 未切片的表格行：斜纹底 + 虚线左边，提示这里是缺口，可以拖选/右键新增切片 */
.grid-table tbody tr.row-gap td {
  background: repeating-linear-gradient(
    45deg,
    rgba(148, 163, 184, 0.16) 0 5px,
    rgba(148, 163, 184, 0.03) 5px 10px
  );
}
.grid-table tbody tr.row-gap td:first-child {
  border-left: 3px dashed #94a3b8;
}
.grid-table tbody tr.row-gap:hover td {
  background: rgba(245, 158, 11, 0.14);
}
/* 选中的表格行（整行行选择器：拖动 / Shift 连选） */
.grid-table tbody tr { user-select: none; }
.grid-table tbody tr.sel-row td {
  background: rgba(245, 158, 11, 0.20) !important;
  box-shadow: inset 0 0 0 1px #f59e0b;
}
```

```
.grid-table tbody tr.sel-row td:first-child {
  border-left: 3px solid #ea580c;
}
.grid-table tbody tr.sel-row + tr.sel-row td { box-shadow: none; }
body.row-picking,
body.row-picking .grid-table tbody tr { cursor: row-resize; }
body.row-picking { user-select: none; }
/* 切片卡片里的表格正文容器 */
.chunk-content { margin: 6px 0 4px; }
.chunk-content .chunk-body { margin: 0; max-height: 240px; }
.chunk-card.active .chunk-content .chunk-body { max-height: 520px; background: #fafdff; }
.chunk-content .tblwrap { max-height: 240px; }
.chunk-card.active .chunk-content .tblwrap { max-height: 520px; }
.chunk-raw { margin-top: 6px; }
.chunk-raw summary {
  color: var(--accent);
  font-size: 11.5px;
  cursor: pointer;
  user-select: none;
}
.chunk-raw summary:hover { text-decoration: underline; }
.chunk-raw-pre {
  margin: 5px 0 0;
  padding: 6px 8px;
  font-family: var(--mono);
  font-size: 11px;
  line-height: 1.55;
  white-space: pre-wrap;
  word-break: break-word;
  background: #fbfcfe;
  border: 1px dashed #e3e7ef;
  border-radius: 6px;
  max-height: 200px;
  overflow: auto;
}
/* 编辑后重渲染时由 JS 精确恢复滚动位置，关闭浏览器自动滚动锚定，避免几像素漂移 */
#source, #chunks { overflow-anchor: none; }
/* 自动切块状态 chip（顶部工具栏） */
.watch-chip {
  font-size: 12px;
  line-height: 1.6;
  padding: 1px 9px;
  border-radius: 999px;
  border: 1px solid var(--border);
  color: var(--muted);
  background: var(--panel);
  white-space: nowrap;
  max-width: 300px;
  overflow: hidden;
  text-overflow: ellipsis;
```

```
}
.watch-chip.ok { color: #0f766e; border-color: #b7e4d8; background: #e8f7f3; }
.watch-chip.warn { color: #92400e; border-color: #fcd34d; background: #fef3c7; }
.watch-chip.err { color: #b91c1c; border-color: #fecaca; background: #fef2f2; }
/* 二选一/三选一弹层（删除分片 part 时用） */
.choice-dialog .choice-actions { display: flex; gap: 8px; justify-content: flex-end; flex-wrap
: wrap; margin-top: 12px; }
.choice-dialog .btn { min-width: 96px; }
/* Ctrl/⌘ 多选卡片 */
.chunk-card.multi-sel { outline: 2px solid #a855f7; outline-offset: -2px; background: #faf5ff;
 }
/* 左栏批量导入原文件 */
.btn-upload { padding: 6px 10px; font-size: 12px; white-space: nowrap; }
.drop-hint {
  margin: 6px 10px 0;
  padding: 8px 10px;
  border: 1px dashed #60a5fa;
  border-radius: 8px;
  background: #eff6ff;
  color: #1d4ed8;
  font-size: 12px;
  text-align: center;
}
.sidebar.dragging { outline: 3px dashed #2563eb; outline-offset: -3px; background: #f8fbff; }
/* 混合块（正文 + 表格）：段落文字在上、块内表格网格在下 */
.mixed-table { margin: 10px 0 4px; }
.mixed-table .grid-table { margin-top: 4px; }
.mixed-table-head { font-size: 12px; color: #64748b; margin: 10px 0 4px; }
.mixed-table .tblwrap { max-height: 320px; overflow: auto; }
/* ---- P3 审核状态三态点（红=未审核、黄=审核中、绿=已审核） ---- */
.review-dot {
  width: 8px; height: 8px; border-radius: 50%; display: inline-block;
  margin-right: 6px; flex: none; border: 1px solid rgba(0, 0, 0, .08);
}
.review-dot.review-none { background: #ef4444; }
.review-dot.review-in_review { background: #f59e0b; }
.review-dot.review-reviewed { background: #16a34a; }
/* ---------- 文件删除模式（P3） ---------- */
#btn-delete-mode.active { background: #fee2e2; border-color: #ef4444; color: #991b1b; }
.sidebar.delete-mode .review-dot { display: none; }
.delete-check { position: absolute; top: 9px; right: 9px; width: 16px; height: 16px;
  border-radius: 50%; border: 2px solid #94a3b8; background: #fff; color: #fff;
  display: inline-flex; align-items: center; justify-content: center;
  font-size: 11px; line-height: 1; }
.delete-check.checked { background: #2563eb; border-color: #2563eb; }
.file.selected-for-delete { background: #eff6ff; border-color: #bfdbfe; }
.delete-bar { position: sticky; bottom: 0; z-index: 5; display: flex; flex-direction: column;
  gap: 6px; padding: 8px 10px; background: #fff; border-top: 1px solid #e2e8f0;
  box-shadow: 0 -6px 12px rgba(15, 23, 42, .06); font-size: 12px; color: #334155; }
.delete-bar-actions { display: flex; gap: 8px; justify-content: flex-end; }
```

```
.btn-danger { background: #dc2626; border-color: #dc2626; color: #fff; }
.btn-danger:hover { background: #b91c1c; border-color: #b91c1c; }
.delete-bar .btn-danger:disabled { opacity: .5; cursor: not-allowed; }
/* 拖动分栏期间冻结两栏交互，减少重排/悬停计算造成的抖动 */
body.resizing #source, body.resizing #chunks, body.resizing .sidebar { pointer-events: none; }
body.resizing { user-select: none; }
.resizer { touch-action: none; }
#confirm-text { white-space: pre-line; }
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>SplitChunkAssistant</title>
  <link rel="stylesheet" href="/static/style.css?v=0.3.43-p3q">
</head>
<body>
  <header class="topbar">
    <div class="brand">
      <span class="logo" aria-hidden="true">SC</span>
      <h1>SplitChunkAssistant</h1>
      <span class="phase" id="phase">P1</span>
    </div>
    <button id="btn-sidebar" class="btn btn-icon" type="button" title="收起文件栏（Ctrl+B）" a
ria-label="收起或展开文件栏" aria-pressed="false">☰</button>
    <div class="status" id="status">加载中…</div>
    <div class="edit-tools">
      <span class="draft-badge" id="draft-badge" hidden></span>
      <span class="dirty" id="dirty" hidden>● 未保存</span>
      <span class="diff" id="diff" title="相对自动切块基线：+ 新增 ~ 修改 - 删除">+0 ~0 -0</sp
an>
      <button id="btn-undo" type="button" class="btn" title="撤销（Ctrl+Z）" disabled>↶ 撤销</
button>
      <button id="btn-redo" type="button" class="btn" title="重做（Ctrl+Shift+Z）" disabled>↷
重做</button>
    </div>
    <div class="actions">
      <span class="watch-chip" id="watch-chip" hidden></span>
      <button id="btn-save-draft" type="button" class="btn" title="把当前编辑存成草稿（Ctrl+S
），稍后再回来继续">保存草稿</button>
      <button id="btn-commit" type="button" class="btn btn-primary" title="提交草稿：校验基线
→ 备份 → 覆盖正式 chunks.jsonl" disabled>提交</button>
      <button id="btn-discard" type="button" class="btn" title="放弃草稿，回到自动切块基线" di
sabled>放弃</button>
      <button id="btn-export" type="button" class="btn" title="导出当前（含未提交修改）的 chun
ks.jsonl" disabled>导出</button>
      <button id="btn-chunk" type="button" class="btn" title="扫描 data/source 里还没切块的文
件并调用 SPLITCHUNK 切块">⚙ 切块</button>
      <button id="btn-refresh" type="button" class="btn">刷新</button>
      <button id="btn-settings" type="button" class="btn">⚙ 设置</button>
```

```
    </div>
  </header>
  <main class="layout">
    <aside class="sidebar" aria-label="文件列表" id="sidebar">
      <div class="search-row">
        <input id="search" type="search" placeholder="搜索法规名 / 分组…" autocomplete="off" a
ria-label="搜索法规">
        <button id="btn-upload" class="btn btn-upload" type="button" title="批量导入原文件（.m
d / .markdown / .txt，写入 data/source）">⬆ 添加文件</button>
        <button id="btn-delete-mode" class="btn btn-upload" type="button" title="删除文件（可
多选）" aria-pressed="false">🗑 </button>
        <input id="file-input" type="file" multiple accept=".md,.markdown,.txt" hidden>
      </div>
      <div class="drop-hint" id="drop-hint" hidden>松开导入到 data/source</div>
      <div class="stats" id="stats"></div>
      <nav class="tree" id="tree" aria-label="文件列表"></nav>
      <div class="delete-bar" id="delete-bar" hidden>
        <span id="delete-bar-text">已选 0 个文件</span>
        <div class="delete-bar-actions">
          <button id="btn-delete-cancel" type="button" class="btn">取消</button>
          <button id="btn-delete-confirm" type="button" class="btn btn-danger" disabled>删除所
选</button>
        </div>
      </div>
    </aside>
    <section class="source-pane" aria-label="原文">
      <header class="pane-head">
        <div class="pane-title" id="source-title">原文</div>
        <div class="pane-meta" id="source-meta">选择左侧文件开始</div>
        <label class="lane-toggle" title="把每个切片覆盖的原文行范围画成左侧竖条（点击竖条可选
中切片）">
          <input type="checkbox" id="toggle-lanes" checked> 范围条
        </label>
      </header>
      <div class="source-tip" id="source-tip">
        在原文<b>点一下</b>会出现蓝色拆分光标（上方提示可拆行/字符）；<b>右键</b>即可编辑此切
片<b>正文/条号</b>、拆分、合并、删除、新增；右键落在标题/目录等没有切片的行，也可以「用这一行
新增切片」。
      </div>
      <div class="source" id="source">
        <div class="empty">
          <h2>原文 ↔ 切片</h2>
          <p>点击左侧任意一份文件，这里会显示 Markdown 原文行；右栏是自动切好的切片。
             <b>悬停</b>任一侧都会高亮另一侧对应的区域；
             <b>在原文里拖选一段文字</b>，右侧覆盖该范围的切片会被高亮；
            左侧<b>彩色竖条</b>是每个切片覆盖的行范围（悬停看切片 id，点击选中）。</p>
        </div>
      </div>
    </section>
    <section class="chunks-pane" aria-label="切片">
```

```
      <div class="resizer" id="resizer" role="separator" aria-orientation="vertical" title="拖
动调整切片栏宽度；双击恢复默认"></div>
      <header class="pane-head">
        <div class="pane-title">切片 <span class="count" id="chunk-count"></span></div>
        <div class="pane-meta" id="chunk-meta"></div>
        <div class="pane-actions">
          <button id="btn-add" type="button" class="btn btn-small" title="在选中切片之后新增一
块（Ctrl+Shift+N）">＋ 新增</button>
          <button id="btn-reset" type="button" class="btn btn-small" title="丢弃本文件的全部手
工修改，回到自动切块结果">↺ 重置</button>
        </div>
      </header>
      <div class="chunks-hint" id="chunks-hint" hidden>
       手工修改只在内存里（刷新页面不丢，关服务才丢）；P3 才落盘。Ctrl+Z 可撤销。
      </div>
      <div class="chunks" id="chunks">
        <div class="empty"><p>尚未选择文件。</p></div>
      </div>
    </section>
  </main>
  <dialog id="settings-dialog" class="dialog">
    <form method="dialog" id="settings-form">
      <h2>⚙ 设置</h2>
      <p class="hint">唯一数据源是本地「两目录」目录：<code>source/</code> 放原文件（可带子目
录分组），
        <code>chunks/</code> 放同名 <code>.chunks.jsonl</code>（可选同名 <code>.extras.jsonl</
code>）。
        修改后保存立即生效（无需重启）。</p>
      <label class="field">
        <span>本地数据目录 <code>data_root</code>（下含 <code>source/</code> 与 <code>chunks/<
/code>）</span>
        <input id="cfg-data-root" type="text" placeholder="例如 data 或 D:\chunk-data" spellch
eck="false">
      </label>
      <label class="field">
        <span>端口 <code>port</code>（重启后生效）</span>
        <input id="cfg-port" type="number" min="1" max="65535">
      </label>
      <label class="field inline-field">
        <span>自动分 part <code>auto_part</code>（超过阈值的切片在加载时自动拆分；表格只按行区
间拆）</span>
        <input id="cfg-auto-part" type="checkbox">
      </label>
      <label class="field">
        <span>自动拆分阈值 <code>max_chars</code>（默认 800，≥200）</span>
        <input id="cfg-max-chars" type="number" min="200" step="50">
      </label>
      <label class="field">
        <span>最小 part 长度 <code>min_part_chars</code>（默认 200，仅文本 part 用）</span>
        <input id="cfg-min-part-chars" type="number" min="1">
```

```
      </label>
      <label class="field inline-field">
        <span>自动切块 <code>auto_chunk</code>（探测到 source 里新文件就用 SPLITCHUNK 切块）</
span>
        <input id="cfg-auto-chunk" type="checkbox">
      </label>
      <label class="field">
        <span>切块引擎目录 <code>chunker_root</code>（留空=自动找 vendor/splitchunk）</span>
        <input id="cfg-chunker-root" type="text" placeholder="留空自动探测" spellcheck="false"
>
      </label>
      <label class="field">
        <span>轮询间隔 <code>chunker_poll</code>（秒，1–60，默认 3）</span>
        <input id="cfg-chunker-poll" type="number" min="1" max="60">
      </label>
      <div class="dialog-errors" id="cfg-errors" hidden></div>
      <div class="dialog-actions">
        <button type="button" id="btn-cancel" class="btn">取消</button>
        <button type="submit" id="btn-save" class="btn btn-primary">保存</button>
      </div>
    </form>
  </dialog>
  <dialog id="split-dialog" class="dialog op-dialog">
    <form method="dialog" id="split-form">
      <h2>✂ 拆分切片</h2>
      <p class="hint">选择在<b>哪一行之前</b>切开：上半块保留原 id 与原条号，下半块获得新 id（
max_seq+1）。
       单行块按字符位置拆。拆完若下半块条号不同，可在下面直接填，或之后用「改」逐个改。</p>
      <div class="op-target" id="split-target"></div>
      <div class="split-lines" id="split-lines"></div>
      <label class="field" id="split-char-row" hidden>
        <span>切分字符位置 <code>at_char</code>（1 ~ 正文长度-1）</span>
        <input id="split-char" type="number" min="1" step="1">
      </label>
      <label class="field">
        <span>下半块条号 <code>law_num</code>（可选；留空会自动识别切点处的条号「第X条」或「5.
4.1」，识别不到则继承原块）</span>
        <input id="split-law-num" type="text" spellcheck="false" placeholder="例如：第二条 或
5.2.1">
      </label>
      <div class="dialog-errors" id="split-errors" hidden></div>
      <div class="dialog-actions">
        <button type="button" class="btn btn-cancel-op">取消</button>
        <button type="submit" id="btn-split-ok" class="btn btn-primary">确认拆分</button>
      </div>
    </form>
  </dialog>
  <dialog id="edit-dialog" class="dialog op-dialog">
    <form method="dialog" id="edit-form">
      <h2>✎ 编辑切片</h2>
```

```
      <p class="hint">改正文会重算 <code>token_len</code> / <code>chunk_hash</code> 并标记 manual，与原文区间脱钩。</p>
      <div class="op-target" id="edit-target"></div>
      <label class="field">
        <span>正文 <code>chunk</code></span>
        <textarea id="edit-text" rows="10" spellcheck="false"></textarea>
      </label>
      <label class="field">
        <span>条号 / 表号 <code>law_num</code></span>
        <input id="edit-law-num" type="text" spellcheck="false">
      </label>
      <label class="field">
        <span>类型 <code>chunk_kind</code></span>
        <select id="edit-kind"></select>
      </label>
      <div class="dialog-errors" id="edit-errors" hidden></div>
      <div class="dialog-actions">
        <button type="button" class="btn btn-cancel-op">取消</button>
        <button type="submit" id="btn-edit-ok" class="btn btn-primary">保存修改</button>
      </div>
    </form>
  </dialog>
  <dialog id="add-dialog" class="dialog op-dialog">
    <form method="dialog" id="add-form">
      <h2 id="add-title">＋ 新增切片</h2>
      <p class="hint" id="add-desc">在原文里拖选一段文字后点「新增」，选区会直接变成切片并记录行区间；未选区时手动输入正文，插入到当前选中切片之后（未选中则追加末尾），id = 当前最大 seq+1。</p>
      <p class="op-hint" id="add-hint" hidden></p>
      <label class="field">
        <span>正文 <code>chunk</code></span>
        <textarea id="add-text" rows="8" spellcheck="false" placeholder="粘贴或输入要新增的块正文…"></textarea>
      </label>
      <label class="field">
        <span>条号 / 表号 <code>law_num</code></span>
        <input id="add-law-num" type="text" spellcheck="false" placeholder="留空则填「（新增）」">
      </label>
      <label class="field">
        <span>类型 <code>chunk_kind</code></span>
        <select id="add-kind"></select>
      </label>
      <div class="dialog-errors" id="add-errors" hidden></div>
      <div class="dialog-actions">
        <button type="button" class="btn btn-cancel-op">取消</button>
        <button type="submit" id="btn-add-ok" class="btn btn-primary">新增</button>
      </div>
    </form>
  </dialog>
```

```
  <dialog id="choice-dialog" class="dialog op-dialog choice-dialog">
    <form method="dialog">
      <h2 id="choice-title">请选择</h2>
      <p class="hint" id="choice-text"></p>
      <div class="choice-actions" id="choice-actions"></div>
    </form>
  </dialog>
  <dialog id="confirm-dialog" class="dialog op-dialog">
    <form method="dialog">
      <h2 id="confirm-title">确认删除？</h2>
      <p class="hint" id="confirm-text"></p>
      <div class="dialog-actions">
        <button id="btn-confirm-cancel" type="button" class="btn">取消</button>
        <button id="btn-confirm-ok" type="button" class="btn btn-danger">确认删除</button>
      </div>
    </form>
  </dialog>
  <div id="toast" class="toast" hidden></div>
  <div id="split-caret" class="split-caret" hidden aria-hidden="true"></div>
  <div id="split-hint" class="split-hint" hidden aria-live="polite"></div>
  <div id="ctx-menu" class="ctx-menu" role="menu" aria-label="切片操作" hidden></div>
  <script src="/static/app.js?v=0.3.43-p3q" defer></script>
</body>
</html>
"""PyInstaller 打包入口（Windows）。
统一走 ``chunk_studio.launcher``：普通启动=本地 Web 服务，
``--run-chunk-cli``=转发给打包进来的 SPLITCHUNK 引擎。
"""
import sys
from chunk_studio.launcher import main
if __name__ == "__main__":
    raise SystemExit(main())
# -*- mode: python ; coding: utf-8 -*-
"""PyInstaller onedir 打包配置（Windows）。
用法（在仓库根）：
    python -m PyInstaller --clean --noconfirm packaging/SplitChunkAssistant.spec
产物：``dist/SplitChunkAssistant/SplitChunkAssistant.exe`` + ``_internal/``。
关键点：
- ``web/`` 与 ``vendor/splitchunk/`` 作为**数据目录**放进 ``_internal``，运行时
  ``chunk_studio.paths`` 从 ``sys._MEIPASS`` 找；引擎从磁盘目录导入，升级引擎
  只要替换 ``_internal/vendor/splitchunk``，不必重新打包。
- 引擎依赖 PyYAML（含 ``_yaml`` C 扩展），交给 PyInstaller 的 yaml hook。
- 配置与 data/ 落在 exe 同级（不可写时回退 %LOCALAPPDATA%），不打进安装包。
"""
import os
ROOT = os.path.abspath(os.path.join(SPECPATH, ".."))  # noqa: F821 - SPECPATH 由 PyInstaller
注入
datas = [
    (os.path.join(ROOT, "web"), "web"),
    (os.path.join(ROOT, "vendor", "splitchunk"), os.path.join("vendor", "splitchunk")),
```

```
]
a = Analysis(  # noqa: F821
    [os.path.join(SPECPATH, "run_app.py")],  # noqa: F821
    pathex=[ROOT],
    binaries=[],
    datas=datas,
    hiddenimports=["yaml"],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=["tkinter"],
    noarchive=False,
    optimize=0,
)
pyz = PYZ(a.pure)  # noqa: F821
exe = EXE(  # noqa: F821
    pyz,
    a.scripts,
    [],
    exclude_binaries=True,
    name="SplitChunkAssistant",
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=False,
    console=True,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
)
coll = COLLECT(  # noqa: F821
    exe,
    a.binaries,
    a.datas,
    strip=False,
    upx=False,
    name="SplitChunkAssistant",
)
; Inno Setup script for SplitChunkAssistant (onedir build).
; Prerequisites:
;   1) Run packaging\build.ps1 so that dist\SplitChunkAssistant\ exists.
;   2) Install Inno Setup 6 (https://jrsoftware.org/isinfo.php).
; Build:
;   "C:\Program Files (x86)\Inno Setup 6\ISCC.exe" packaging\installer.iss
; Output: dist\SplitChunkAssistant-Setup.exe
;
; Notes:
; - Default is a per-user install (no admin): {autopf} resolves to
```

```
;   %LOCALAPPDATA%\Programs\SplitChunkAssistant. Users may still choose an
;   all-users install from the privileges dialog.
; - config.json and data\ are NOT bundled: on first run the app creates them next
;   to the exe, or falls back to %LOCALAPPDATA%\SplitChunkAssistant when the
;   install directory is not writable.
; - Bump MyAppVersion for each release.
#define MyAppName "SplitChunkAssistant"
#define MyAppVersion "0.3.43"
#define MyAppPublisher "SplitChunkAssistant"
#define MyAppExeName "SplitChunkAssistant.exe"
[Setup]
AppId={{8F3B2C1A-6D2E-4C59-9A87-2B4E7D5C1F60}
AppName={#MyAppName}
AppVersion={#MyAppVersion}
AppPublisher={#MyAppPublisher}
DefaultDirName={autopf}\{#MyAppName}
DefaultGroupName={#MyAppName}
DisableProgramGroupPage=yes
OutputDir=..\dist
OutputBaseFilename=SplitChunkAssistant-Setup
Compression=lzma
SolidCompression=yes
ArchitecturesInstallIn64BitMode=x64compatible
UninstallDisplayIcon={app}\{#MyAppExeName}
PrivilegesRequired=lowest
PrivilegesRequiredOverridesAllowed=dialog
WizardStyle=modern
[Languages]
Name: "english"; MessagesFile: "compiler:Default.isl"
[Tasks]
Name: "desktopicon"; Description: "Create a desktop shortcut"; GroupDescription: "Additional i
cons:"
[Files]
Source: "..\dist\SplitChunkAssistant\*"; DestDir: "{app}"; Flags: ignoreversion recursesubdirs
 createallsubdirs
[Icons]
Name: "{group}\{#MyAppName}"; Filename: "{app}\{#MyAppExeName}"
Name: "{group}\Uninstall {#MyAppName}"; Filename: "{uninstallexe}"
Name: "{autodesktop}\{#MyAppName}"; Filename: "{app}\{#MyAppExeName}"; Tasks: desktopicon
[Run]
Filename: "{app}\{#MyAppExeName}"; Description: "Launch {#MyAppName}"; Flags: nowait postinsta
ll skipifsilent
@echo off
rem SplitChunkAssistant launcher (Windows).
rem ASCII only + CRLF line endings - do not add non-ASCII characters to this file,
rem cmd.exe reads batch files with the OEM codepage and UTF-8 text breaks parsing.
rem
rem Usage:
rem   run_studio.cmd                      default port 8790, opens browser
rem   run_studio.cmd --no-open            do not open browser
```

```
rem   run_studio.cmd --port 8801 --chunks-dir "D:\data\chunks_v2"
setlocal
cd /d "%~dp0"
chcp 65001 >nul
set "PYTHONIOENCODING=utf-8"
set "PYTHONUTF8=1"
set "PY="
where python >nul 2>nul && set "PY=python"
if not defined PY (
  where py >nul 2>nul && set "PY=py -3"
)
if not defined PY (
  echo [x] Python not found in PATH.
  echo     Install Python 3.10+ from https://www.python.org/downloads/
  echo     and tick "Add python.exe to PATH" during install.
  pause
  exit /b 2
)
%PY% -c "import sys" >nul 2>nul
if errorlevel 1 (
  echo [x] Python found but cannot run: %PY%
  pause
  exit /b 2
)
%PY% -m chunk_studio.server %*
set "RC=%ERRORLEVEL%"
if not "%RC%"=="0" (
  echo.
  echo [x] Server exited with code %RC%
  echo     If the port is already taken, try: run_studio.cmd --port 8801
  pause
)
endlocal & exit /b %RC%
```